VisionGuard

AI-generated vs real image classification and deployment

MLOps - Assignment 2

Name	Muhammad Sabih ud din khan
Roll number	23L-0654
GitHub	https://github.com/MSabihkhan/VisionGuard
DagsHub	https://dagshub.com/MSabihkhan/VisionGuard
Live API URL	http://18.206.172.160
Commit hash of the graded version	406c59248ee233afac0b20688147fe3162583f7f

Front matter and disclosure

Links

- GitHub repository: https://github.com/MSabihkhan/VisionGuard
- DagsHub project (code, data with DVC, MLflow experiments and model registry): https://dagshub.com/MSabihkhan/VisionGuard
- MLflow tracking address: https://dagshub.com/MSabihkhan/VisionGuard.mlflow
- Model API (AWS EC2, us-east-1): http://18.206.172.160 (endpoints `/health` and `/predict`)
- Graded commit: `406c59248ee233afac0b20688147fe3162583f7f`

Note about the project name

The assignment says the project name should be the roll number followed by A2_Ops. My GitHub and DagsHub project is called **VisionGuard** because I created it before I noticed this rule, so the repository name does not follow it. The Compose project name (`23l-0654_a2_ops`) and the AWS resources (`23l-0654-a2-ops-api`, tag `23L-0654_A2_Ops-api`) do use the roll number.

AI assistant disclosure

I used Claude Code (an AI assistant) to help me write this report. The code in the project I mostly wrote myself. I used the AI assistant only where I did not know the syntax, for example for some Keras, MLflow, Docker, GitHub Actions and AWS command line details, and then I ran and checked everything myself. I ran all the training, took all the screenshots and looked at all the results.

How to read this report

Every chapter follows the task numbers of the assignment. Under each sub-task I write what I did, why, how it works and what went wrong.

Summary of results

Item	Result
Dataset	975 images (539 AI, 436 real), 0 corrupt, 1 exact duplicate
Split (seed 42, stratified)	train 681, validation 146, test 147
Baseline CNN (test)	accuracy 0.667, F1 0.746, ROC-AUC 0.670
MobileNetV2 fine-tuned (test)	accuracy 0.728, F1 0.785, ROC-AUC 0.797
Champion model (EfficientNetB0, run *effnet*, registry version 2)	accuracy 0.755, precision 0.699, recall 0.975, F1 0.814, ROC-AUC 0.856, threshold 0.34
Tests	26 pytest tests pass
API	Hosted on AWS EC2, protected by an API key, tested in Postman

Task 1: Data understanding and versioning

1.1 Environment

What I did. I made a Python virtual environment with `python -m venv .venv`, installed TensorFlow and the other libraries inside it, and saved the exact versions with `pip freeze > requirements.txt` (Figure 1). The main versions are Python 3.14, TensorFlow 2.22.0rc0, Keras 3.15.1, MLflow 3.17.1, Flask 3.1.3, SQLAlchemy 2.1.5 and DVC.

Why. A virtual environment keeps the project libraries separate from the rest of my computer, so the versions are the same every time and I can write them down for someone else.

GPU check. My laptop has no GPU that TensorFlow can use, so everything runs on the CPU (TensorFlow prints the message about CPU instructions such as AVX2 when it starts). This is also why I kept the images small (160 x 160) and the models light.

What went wrong. `pip freeze` writes every package, including ones I do not use, so `requirements.txt` is long. For Docker I wrote a short separate file (`requirements-docker.txt`) with only the packages the app needs.

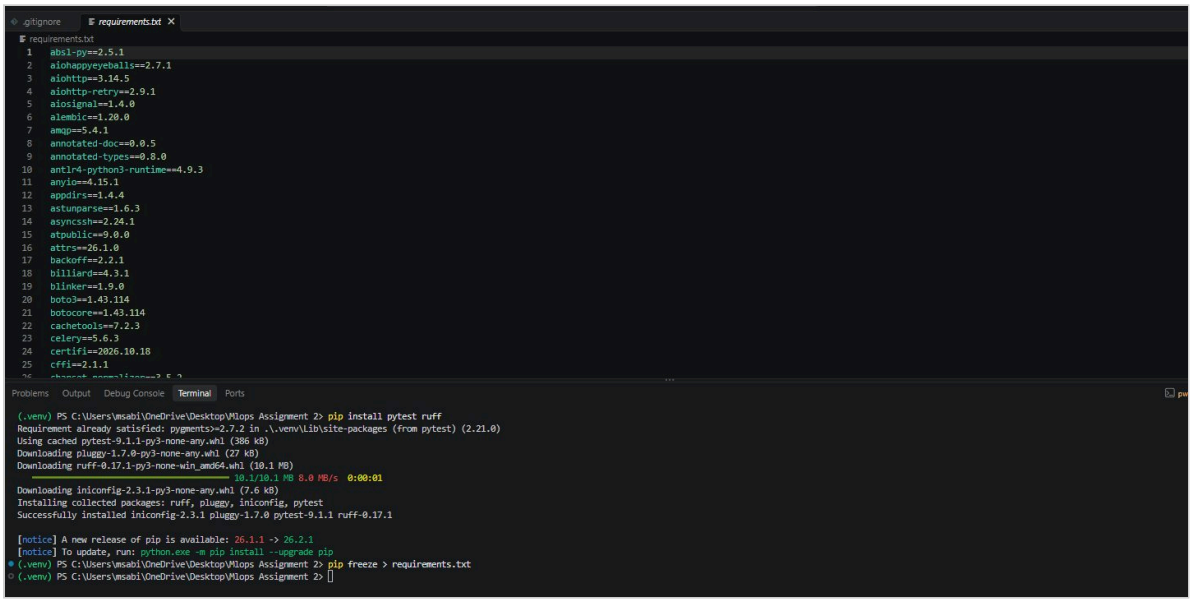


Figure 1. *Virtual environment, installing packages and saving versions with pip freeze.*

1.2 Dataset audit

What I did. I wrote `audit.py`. It walks the `data/raw` folder and for every image records the class (the folder name), file size, width, height, colour mode, format and a SHA-256 hash of the file. It opens each image with Pillow to see if it is readable. Images with the same hash are exact duplicates. The table is saved to `reports/audit.csv` (Figure 2).

Why. I wanted to know if there are broken files, duplicates, or something very different between the two classes before training. Duplicates are dangerous because one copy can end up in train and the other in test, which makes the test score look better than it is.

	AI-generated	Real
Number of images	539	436
Width min / median / max	225 / 1024 / 6000	183 / 920 / 8495
Height min / median / max	135 / 800 / 4001	158 / 720 / 7091
File size min / median / max (bytes)	11,738 / 199,760 / 5,626,183	11,311 / 144,840 / 10,195,352
Formats	JPEG 429, PNG 110	JPEG 403, PNG 33
Colour modes	RGB 489, RGBA 35, P 14, L 1	RGB 407, RGBA 14, P 9, L 6

Corrupt or unreadable files: **0**. Exact duplicates: **1** (it was inside the AI class, no hash appears in both classes). I removed the duplicate when I made the splits, so 974 images are used.

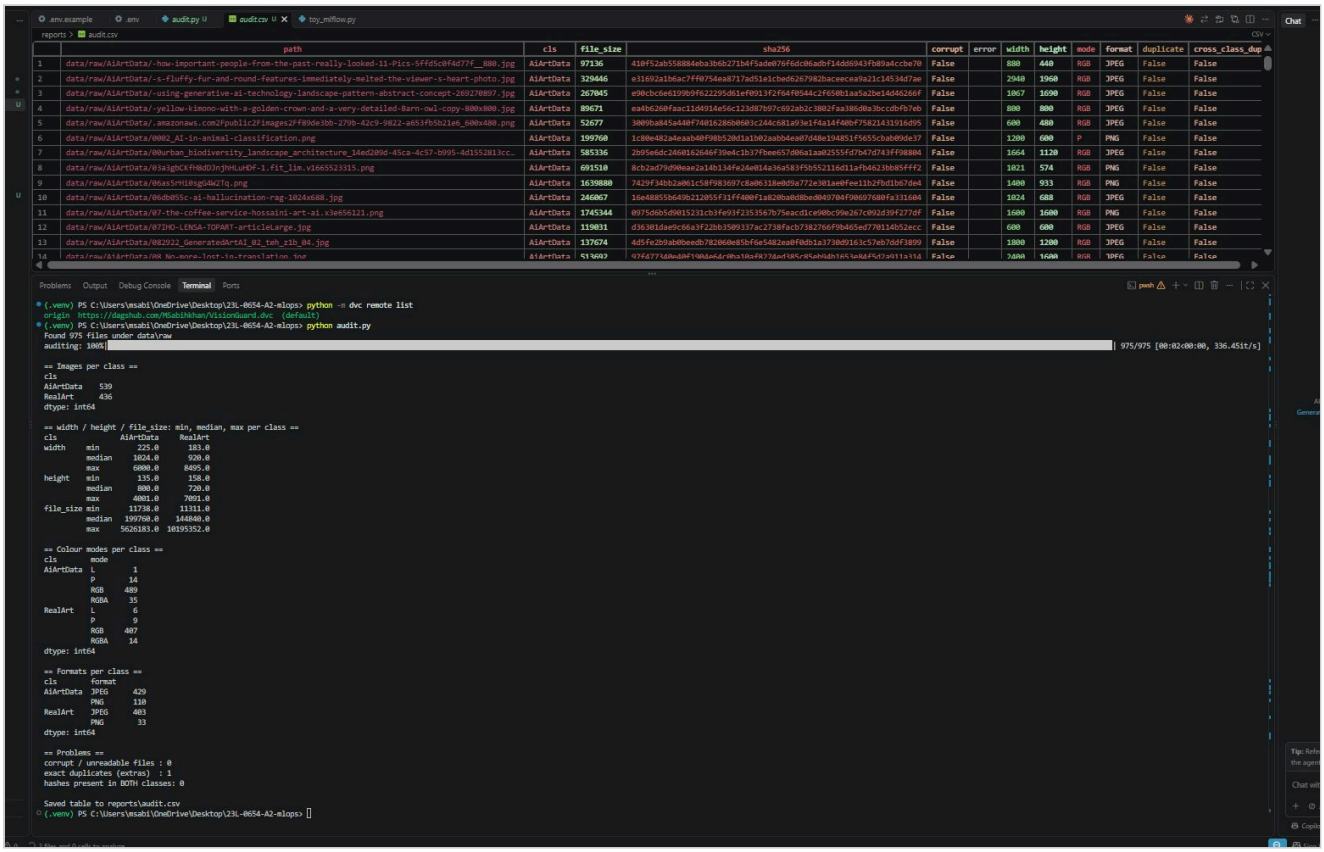


Figure 2. *Output of audit.py in the terminal and the first rows of audit.csv.*

Class balance. The data is slightly unbalanced (about 55% AI). Because of this I use F1 and ROC-AUC and not accuracy alone, and I made the splits stratified.

Splits. `make_splits.py` does a stratified 70 / 15 / 15 split with seed 42 and writes `splits/train.csv`, `val.csv` and `test.csv` (paths and labels). The split sizes are 681, 146 and 147 and the AI share is 0.552, 0.555 and 0.551 in each, so all three have the same class mix (Figure 3).



Figure 3. *make_splits.py output: sizes and share of AI images in each split.*

1.3 Visual EDA

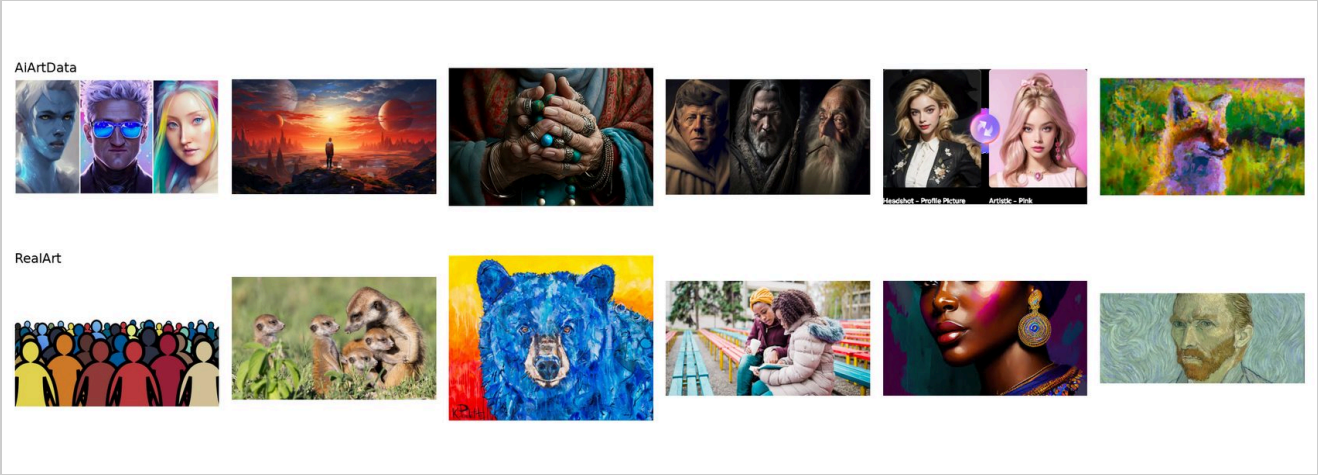


Figure 4. *Six random images from each class. Top row: AI-generated (AiArtData). Bottom row: real (RealArt).*

Interpretation. The AI images look like digital art: portraits with perfect skin, glowing sunsets and fantasy scenes. The real class is not only photos. It also has paintings, cartoons and illustrations (the blue bear, the Van Gogh portrait, the crowd of people). So the label means real artwork or real photo made by a human, and the model cannot just learn that a photo means real. This explains many of the mistakes later in Task 2.6.

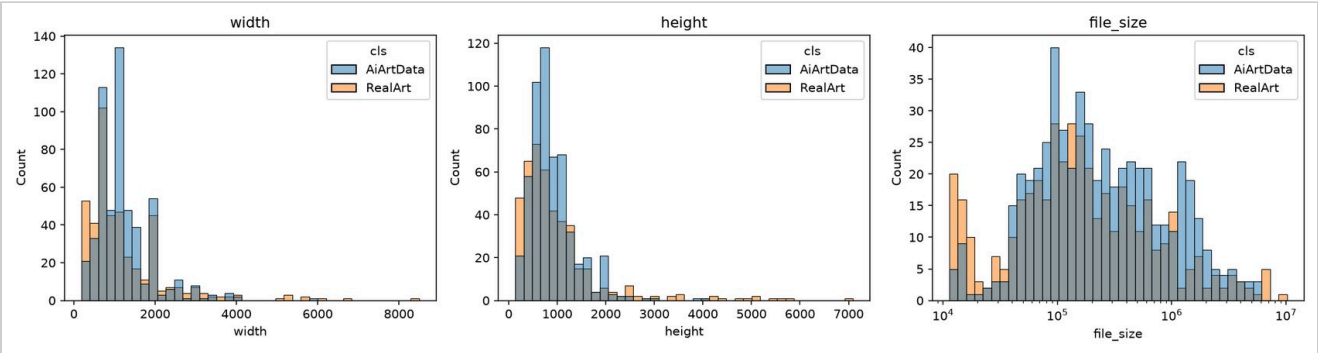


Figure 5. *Histograms of width, height and file size per class.*

Interpretation. Most AI images are around 1000 pixels wide and 600 to 1000 pixels high, with a sharp peak near 1024 (the usual output size of generators), and they have many files between 100 KB and 2 MB. Real images are more spread out: some are tiny (many under 20 KB) and a few are very large (up to 8495 pixels and 10 MB). The size of the file is therefore a weak shortcut the model could use. I resize every image to the same size so the model cannot read the original size, but it can still see compression and sharpness differences.

1.4 DVC (data version control)

Toy part. In a new folder `dvc_toy` I made a small `data.csv` (columns id, value), ran `git init` and `dvc init`, and committed (Figure 6). I connected the folder to a DagsHub repository and pushed. Then I tracked the CSV with `dvc add data.csv`, committed the pointer file as version 1 (Figure 7), added a new row (4,40), ran `dvc add` again and committed version 2 (Figure 8). I configured the DagsHub storage as the DVC remote, ran `dvc push` and pushed the git commits (Figure 9). To prove the remote works I deleted `data.csv` and the local `.dvc/cache` and ran `dvc pull`, which brought the file back (Figure 10). Finally I went back and forth between versions with `git checkout HEAD -- data.csv.dvc` followed by `dvc checkout`. The latest version has four rows (Figure 11) and the older one has three rows (Figure 12).

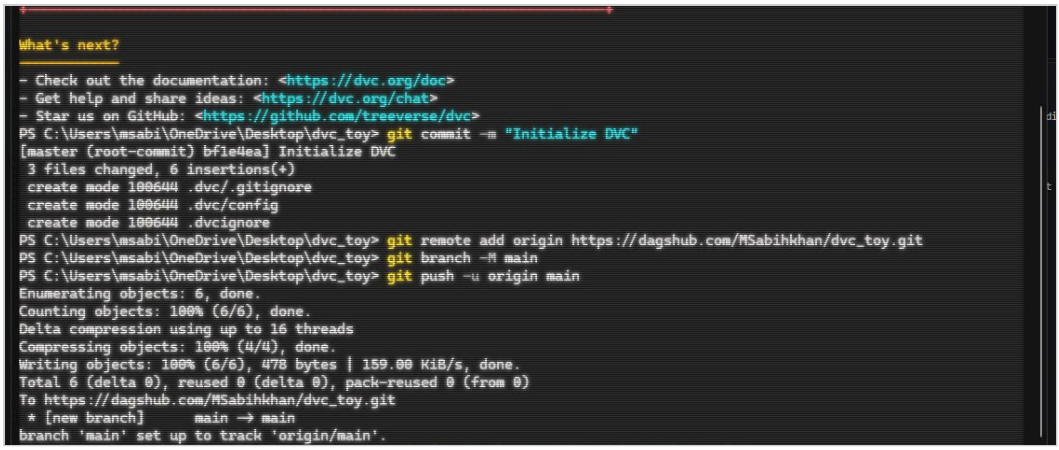


Figure 6. *Toy repository: git commit of DVC init, DagsHub remote added and pushed.*

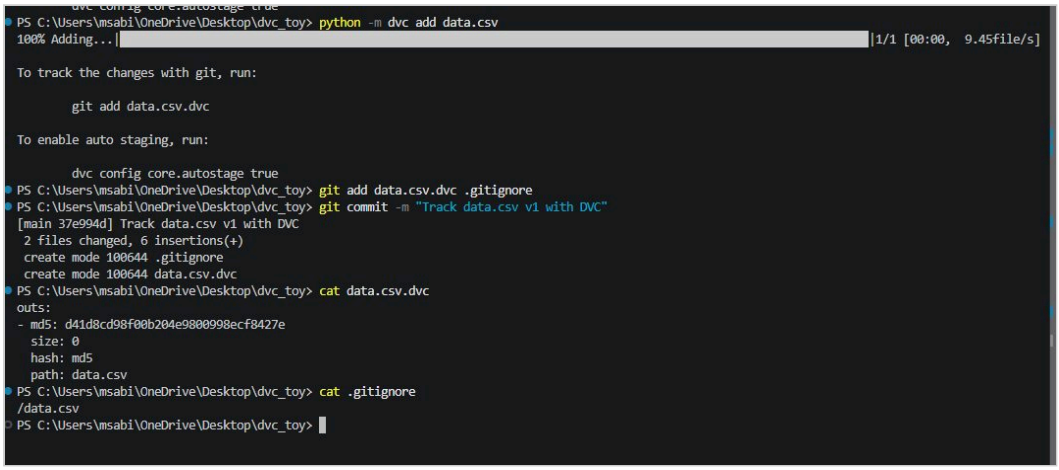


Figure 7. *Version 1 of data.csv tracked with DVC. The pointer file data.csv.dvc and the .gitignore entry are shown.*

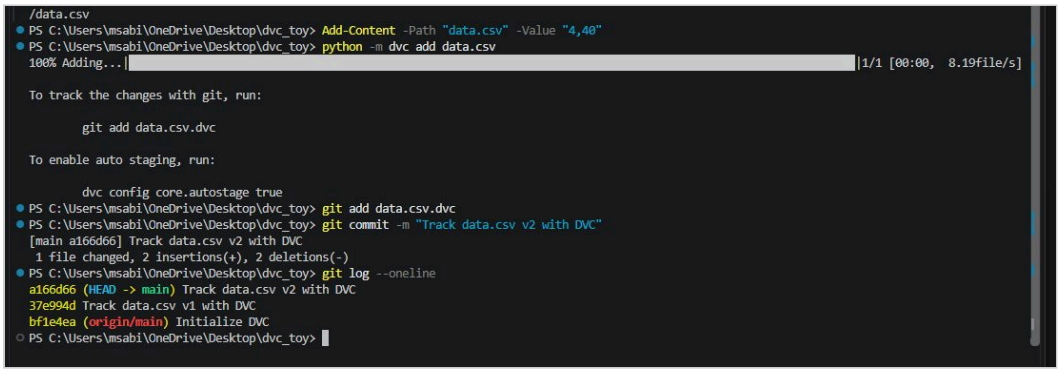


Figure 8. *Version 2 after adding a row, and the git log with both versions.*

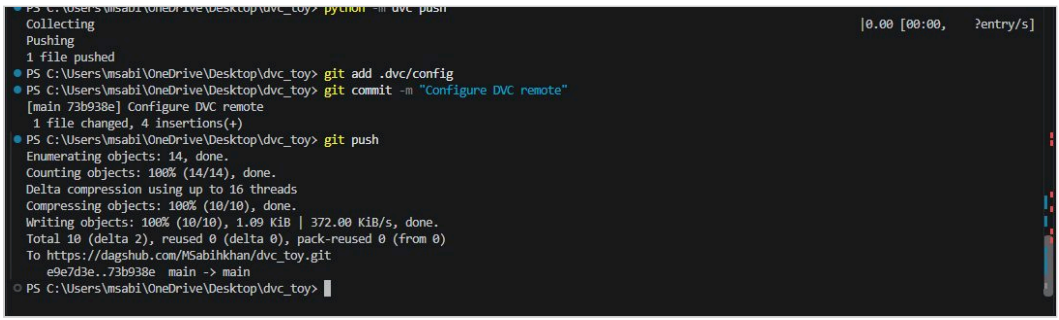


Figure 9. *dvc push to the DagsHub remote, then committing the .dvc/config and pushing to git.*

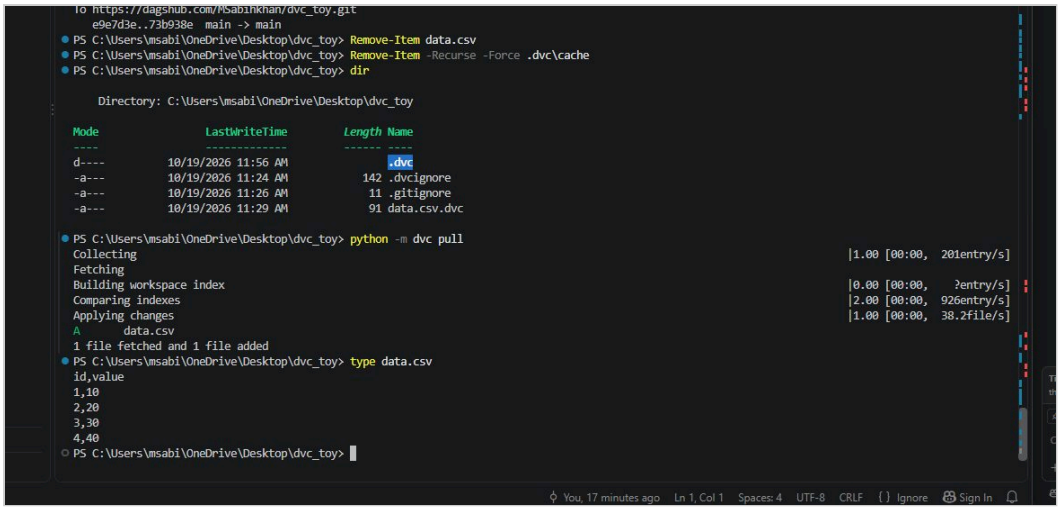


Figure 10. *Local file and cache deleted, then dvc pull restores data.csv with all four rows.*

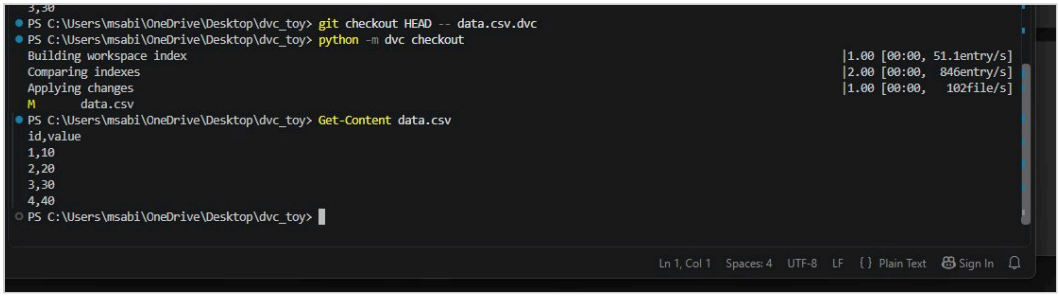


Figure 11. *git checkout of the newest pointer file and dvc checkout: four rows (latest version).*

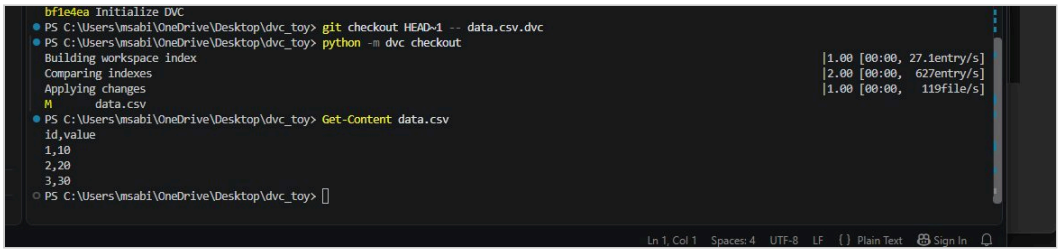


Figure 12. *git checkout HEAD~1 of the pointer file and dvc checkout: three rows (older version).*

What the pointer file holds. `data.csv.dvc` is a few lines of text. It holds the MD5 hash of the data (`md5`), its size, and the path. The real data is stored somewhere else (the DVC cache and the remote) under that hash. Git only stores the small pointer, so every git commit points to an exact version of the data.

Why large data does not belong in Git. Git keeps every version of every file forever, so a big dataset makes the repository huge and slow to clone, and GitHub refuses files over 100 MB. Git is also not good at comparing binary files such as images. With DVC the repository stays small and the data is downloaded only when needed with `dvc pull`.

Apply part. I did the same for the real dataset. I ran `dvc init`, `dvc add data/raw` (975 files), committed `data/raw.dvc`, added the DagsHub storage as remote (`origin`), ran `dvc push` (975 files pushed) and committed the config (Figure 13 and Figure 14).

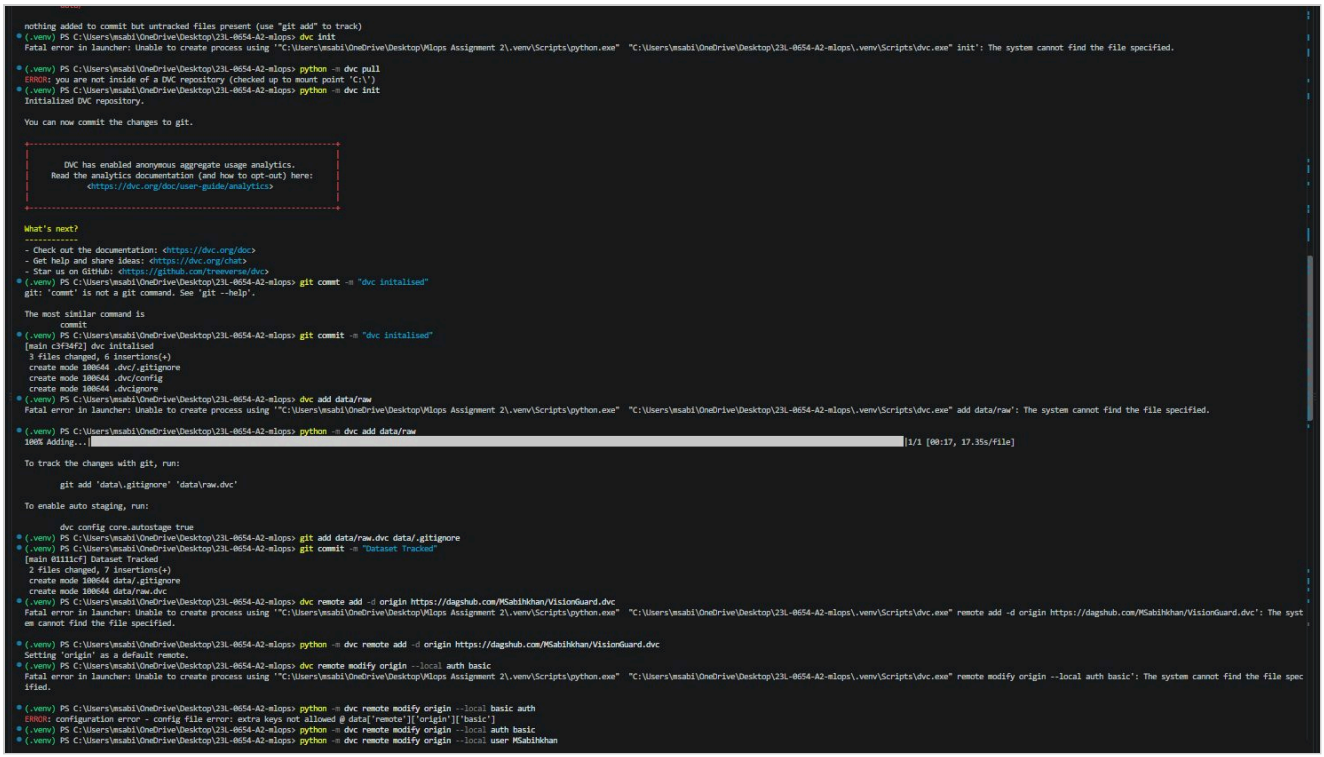


Figure 13. *Real dataset: dvc init, dvc add data/raw and the DagsHub remote setup.*



Figure 14. *Real dataset: dvc push (975 files pushed) and git push.*

What went wrong. The `dvc.exe` in my virtual environment did not run ("Fatal error in launcher ... cannot find the file specified"), because I had moved the project folder. I used `python -m dvc` instead, and it worked. I also first typed `git commt` and `git puhs` and the passwords for the remote are stored with `--local` so they stay in `.dvc/config.local` which git ignores. Later the models folder was also put under DVC (`models.dvc`), so the trained models are versioned without being in git.

Task 2: Model development with TensorFlow/Keras

2.1 Input pipeline

What I did. `data.py` builds a `tf.data` pipeline from the split CSV files:

- read the CSV and make a dataset of (file path, label);
- `map`: read the file bytes and call `prepare()` (decode to 3 channels and resize to 160 x 160), in parallel with `AUTOTUNE`;
- `cache()` so the decoding and resizing is done only once;
- for the training set only: `shuffle` with seed 42 and then random augmentation (horizontal flip, rotation 0.1, zoom 0.1, contrast 0.1);
- `batch(32)` and `prefetch(AUTOTUNE)` so the next batch is prepared while the model trains.

Why. Reading images from disk is slow, so caching and prefetching keep the CPU busy with training. The shuffle comes before augmentation and after the cache so each epoch gets a different order and different random changes. Augmentation is only for training because validation and test must show the real images the model will see in production; if I augmented them the scores would not be comparable.

Image size and aspect ratio. I chose 160 x 160. It is big enough to keep the texture of the images, small enough to train on a CPU in a few minutes, and a valid size for MobileNetV2 and EfficientNetB0. I use plain `tf.image.resize`, so the image is stretched to a square and the aspect ratio is not kept (a wide image becomes squeezed). I decided not to pad or crop because padding adds black borders that look the same in both classes and cropping can remove the part of the image that shows the artifact. The same function is used in the web app, so training and serving see the same pixels.

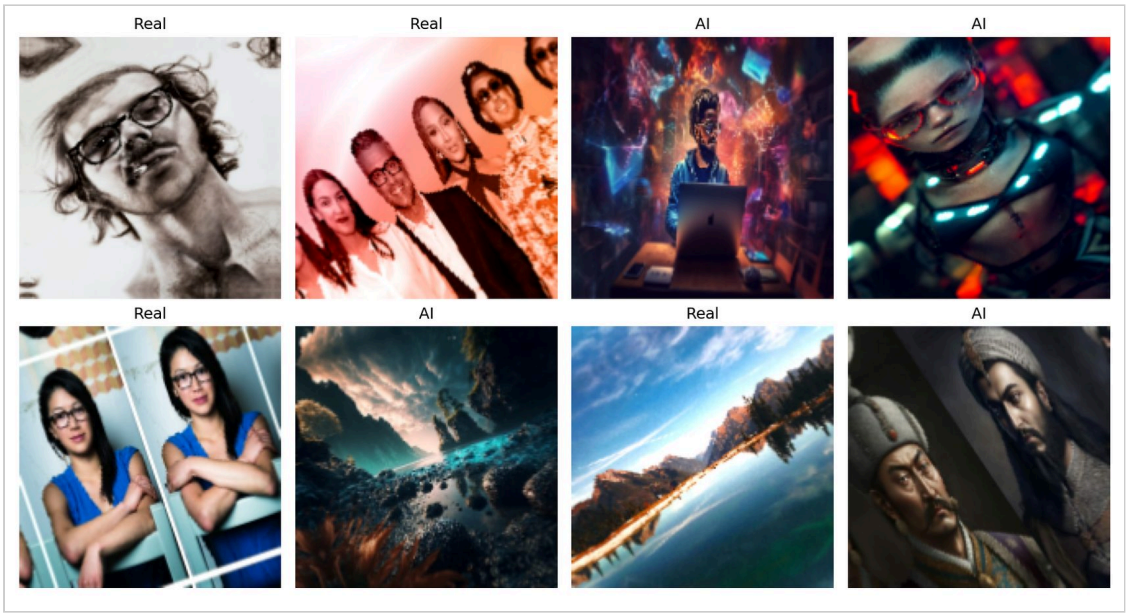


Figure 15. *One augmented training batch of 8 images (labels above each image). Flips, rotations, zoom and contrast changes are visible.*

Design note. The two models need different pixel scaling (the CNN uses 0 to 1, MobileNetV2 expects -1 to 1). So the pipeline returns raw 0 to 255 values and each model does its own scaling in its first layer (see 2.2 and 2.3). This also means the web app does not need to know which model it serves.

2.2 Baseline CNN

What I did. I wrote a small CNN in `train_cnn.py` with three convolution blocks (32, 64 and 128 filters), batch normalization, max pooling, global average pooling, a dense layer of 64, dropout 0.5 and one sigmoid output. It has 102,465 parameters (Figure 16).

Layer	Output shape	Parameters	Why it is there
Rescaling (1/255)	160 x 160 x 3	0	Turns pixels 0-255 into 0-1, built into the model so the app needs no extra step.
Conv2D 32, 3x3	160 x 160 x 32	896	Learns small edges and textures.
BatchNormalization	160 x 160 x 32	128	Keeps activations stable, trains faster.
MaxPooling2D	80 x 80 x 32	0	Halves the size and keeps the strongest signals.
Conv2D 64	80 x 80 x 64	18,496	Combines edges into patterns.
BatchNormalization	80 x 80 x 64	256	Same as above.
MaxPooling2D	40 x 40 x 64	0	Smaller again.
Conv2D 128	40 x 40 x 128	73,856	Learns bigger shapes.
BatchNormalization	40 x 40 x 128	512	Same as above.
MaxPooling2D	20 x 20 x 128	0	Smaller again.
GlobalAveragePooling2D	128	0	One number per filter instead of a huge flatten, much fewer weights, less overfitting.
Dense 64, ReLU	64	8,256	Combines the features.
Dropout 0.5	64	0	Randomly switches off half the units while training to reduce overfitting (only 681 training images).
Dense 1, sigmoid	1	65	Gives a probability that the image is AI-generated.

Total parameters 102,465; trainable 102,017; non-trainable 448 (the batch normalization statistics).

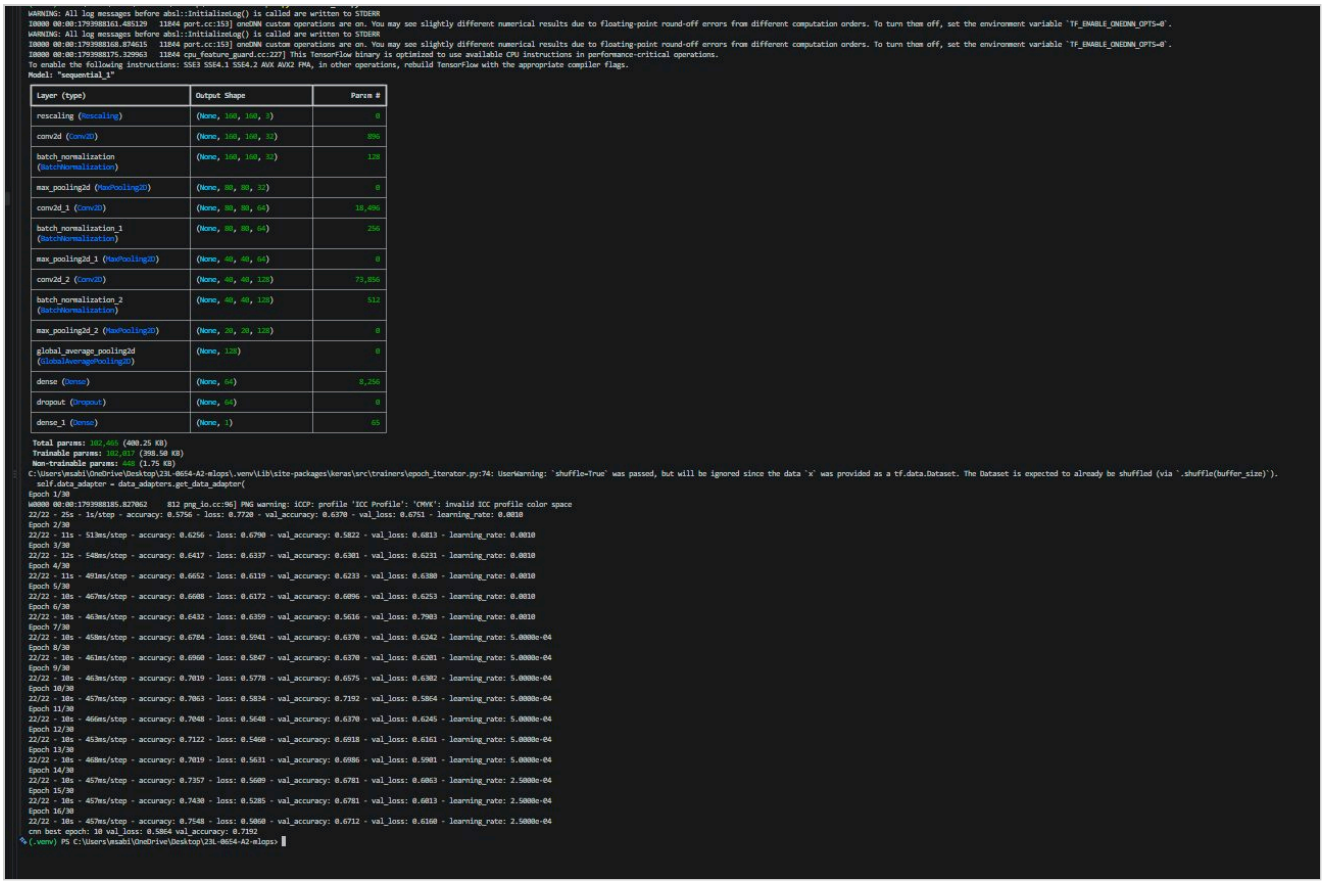


Figure 16. *Model summary of the baseline CNN and the training log (the run stopped early at epoch 16 with the best epoch 10).*

Loss, optimizer and metrics. Loss is **binary cross-entropy** because there are two classes and the output is a probability. Optimizer is **Adam** with learning rate 0.001, a normal default that works well without much tuning. Metrics: accuracy during training, and F1, precision, recall and ROC-AUC in the final evaluation (2.5).

What went wrong. In my first try the CNN predicted the same class for every validation image: the validation accuracy stayed at 0.4452 and the best epoch was epoch 1. The cause was batch normalization. Each epoch has only 22 steps, and with the default momentum (0.99) the moving averages used at test time had not caught up yet. I set `BatchNormalization(momentum=0.9)` and the problem went away.

2.3 Transfer learning

What I did. I loaded MobileNetV2 with ImageNet weights and no classifier (`include_top=False`), froze it, and added my own head: global average pooling, dropout 0.3 and one sigmoid unit. **Stage 1** trains only the head (30 epochs, learning rate 1e-3). **Stage 2** unfreezes only the last block (`block_16_*` and `Conv_1*`) and trains again with learning rate 1e-5, 100 times smaller (up to 15 epochs).

Stage	Trainable layers (with weights)
Stage 1 (head only)	dense (1 layer)
Stage 2 (last block + head)	block_16_expand, block_16_expand_BN, block_16_depthwise, block_16_depthwise_BN, block_16_project, block_16_project_BN, Conv_1, Conv_1_bn, dense (9 layers)

Why. The pretrained model already knows edges, textures and shapes from ImageNet, which helps because I only have 681 training images. First I train only the head, because a randomly started head would send big, wrong gradients into the good pretrained weights. Then I open only the last block with a very small learning rate so it adapts a little to my data without destroying what it learned.

Preprocessing. MobileNetV2 was trained with pixels scaled to the range [-1, 1], so I call `tf.keras.applications.mobilenet_v2.preprocess_input` inside the model. If I used the wrong scaling the

pretrained filters would see inputs they never saw during training and would work badly. The base model is called with `training=False` so its batch normalization layers stay in inference mode even when I unfreeze layers.

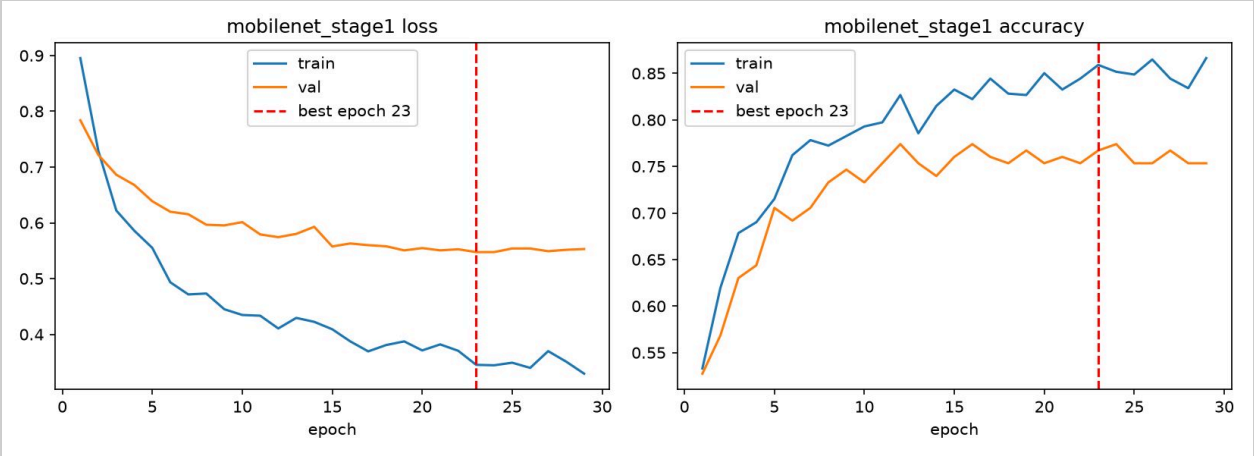


Figure 17. *Training curves of stage 1 (head only). The dashed line is the best epoch (23).*

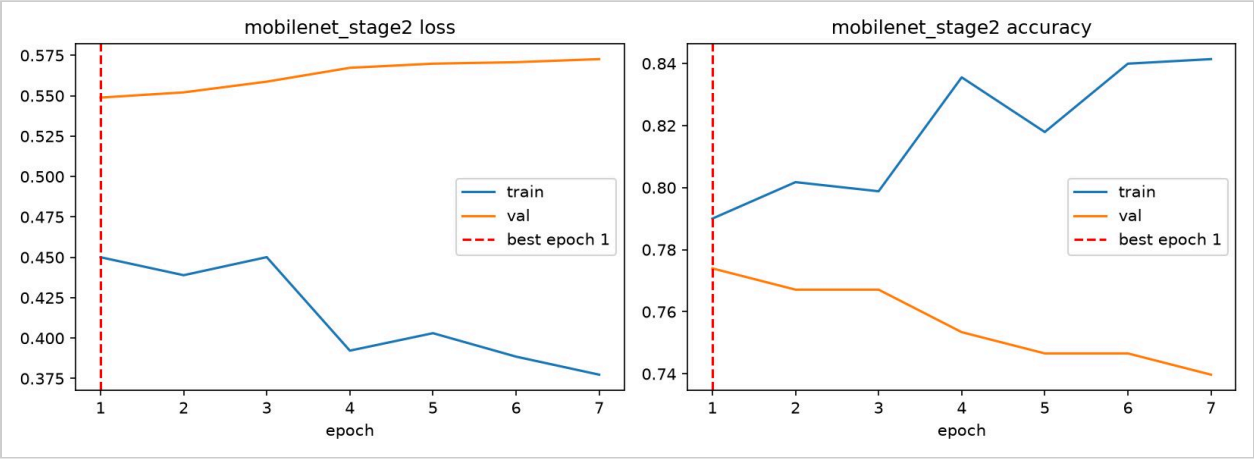


Figure 18. *Training curves of stage 2 (last block unfrozen, learning rate 1e-5).*

What went wrong. With 15 epochs in stage 1 the validation loss was still going down at the end, so I increased it to 30. In stage 2 the fine-tuning did not help: the best epoch was epoch 1 (validation loss 0.5489, compared with 0.5474 at the end of stage 1) and early stopping ended it after 7 epochs. I think the last block starts to overfit the very small training set. I kept the stage 2 model for the comparison because it is the one the assignment asks for, but I note that it is not clearly better.

2.4 Training discipline

What I did. All training scripts call `tf.keras.utils.set_random_seed(42)`. They share three callbacks in `helpers.py`:

- **EarlyStopping** on validation loss, patience 6, with `restore_best_weights=True`: stops when the model stops improving and goes back to the best weights;
- **ModelCheckpoint** on validation loss with `save_best_only=True`: always keeps the best model on disk;
- **ReduceLROnPlateau** (factor 0.5, patience 3): halves the learning rate when the validation loss is stuck.

Model	Epochs run	Best epoch	Validation loss	Validation accuracy
Baseline CNN	16	10	0.5864	0.7192
MobileNetV2 stage 1	29	23	0.5474	0.7671
MobileNetV2 stage 2	7	1	0.5489	0.7740

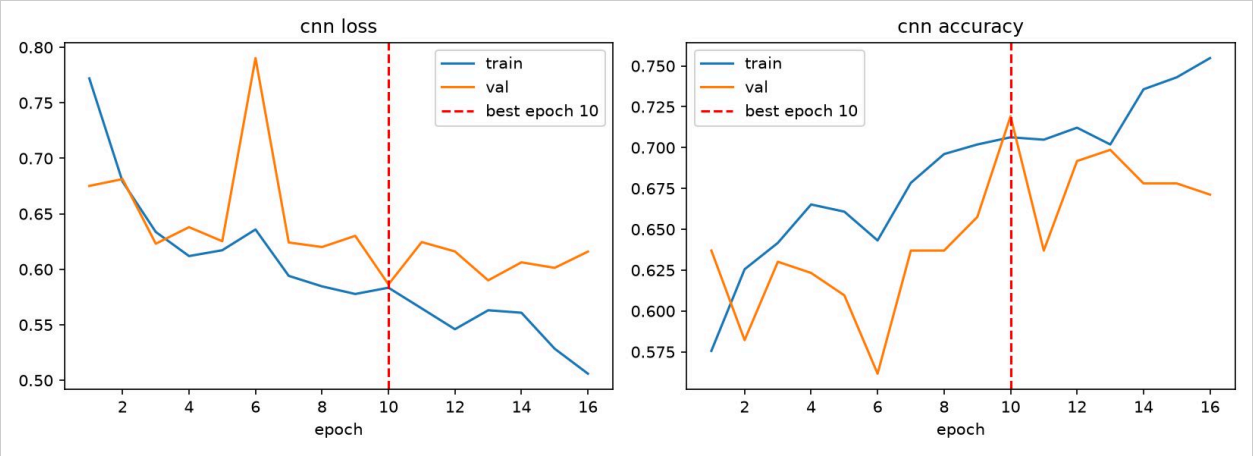


Figure 19. *Loss and accuracy curves of the CNN with the best epoch (10) marked.*

Why. With a small dataset the training accuracy goes up forever while the validation score gets worse. Early stopping and the checkpoint stop this overfitting, and a fixed seed makes the numbers repeatable. The gap between the train and validation curves in the figures (for example train accuracy 0.86 against validation 0.75 for MobileNetV2) still shows some overfitting.

2.5 Evaluation

What I did. `evaluate.py` loads both models and evaluates them on the test set once. The decision threshold is not 0.5: for each model I try thresholds from 0.05 to 0.95 on the **validation** set and take the one with the highest F1 (0.32 for the CNN and 0.31 for MobileNetV2). The test set is only used after the threshold is fixed, so it never influences any choice.

Model	Threshold	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline CNN	0.32	0.667	0.643	0.889	0.746	0.670
MobileNetV2 (fine-tuned)	0.31	0.728	0.695	0.901	0.785	0.797

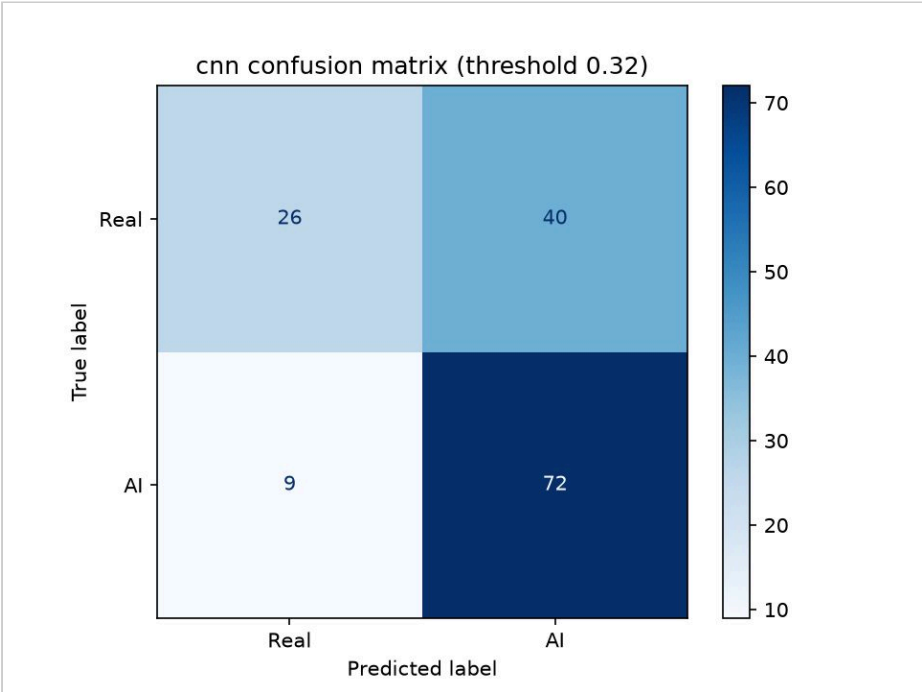


Figure 20. *Confusion matrix of the CNN on the test set (40 real images called AI, 9 AI images called real).*

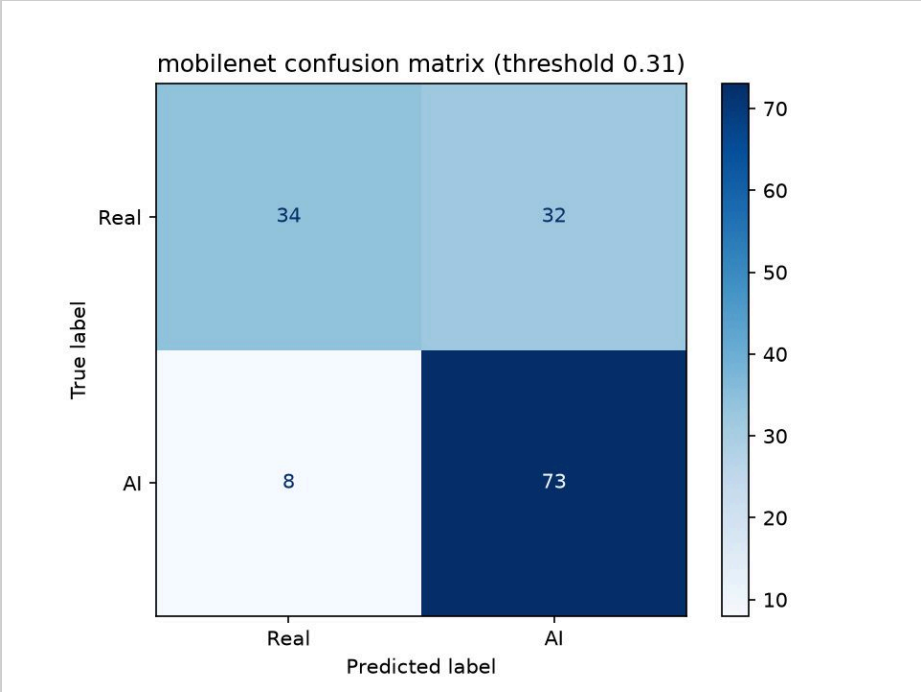


Figure 21. *Confusion matrix of MobileNetV2 on the test set (32 real called AI, 8 AI called real).*

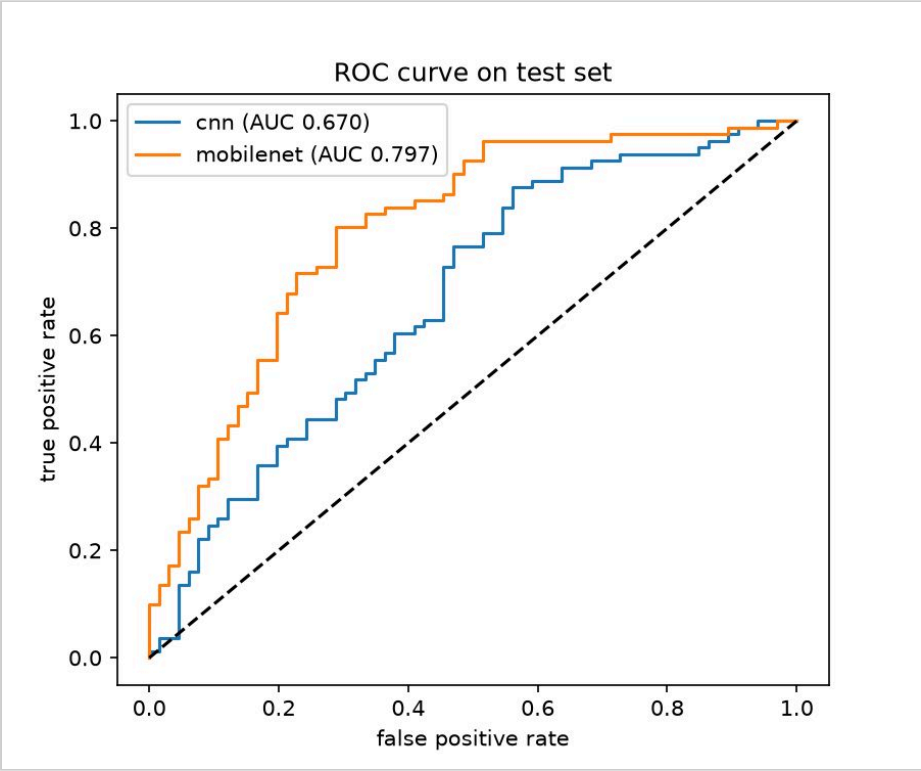


Figure 22. *ROC curves of both models on the test set.*

Result. MobileNetV2 is better than my CNN on every metric (ROC-AUC 0.797 against 0.670), which is expected because it already knows ImageNet features. The gap is big enough that I trust it even with only 147 test images, but the exact numbers can move a few points with another split.

Which error is worse? I think calling a **real photo fake** is the worse error for this product. It accuses a person or a photographer of cheating and it can hurt them, while a fake that slips through is the normal failure of any detector. Maximizing F1 gave a low threshold (about 0.31), so the models call most images AI: recall is 0.90 but 32 of 66 real images are wrongly called fake. If real-to-fake is the worse error, the product should use a higher threshold, accept more missed fakes, and show the verdict as an estimate (the web page has this warning). I kept the F1 threshold in this assignment so the numbers are comparable, but I would change this before a real launch.

2.6 Error analysis

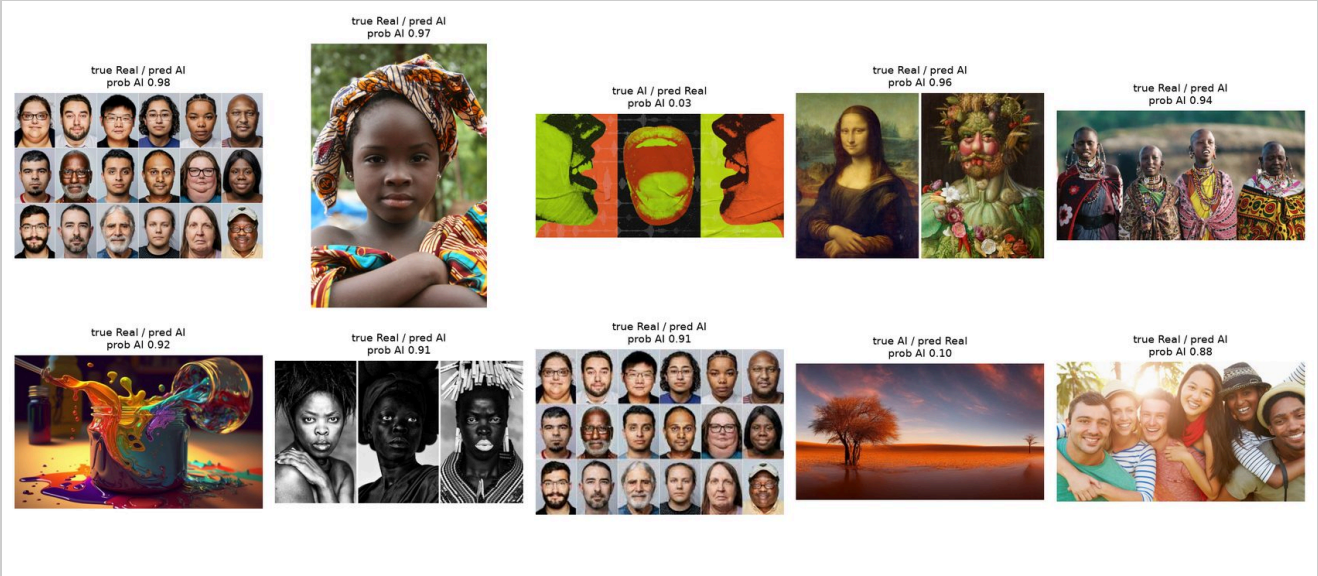


Figure 23. *The 10 most confident mistakes of the MobileNetV2 model on the test set (8 real images predicted as AI, 2 AI images predicted as real). The title shows the true label, the prediction and the AI probability.*

#	Image (row, column)	Hypothesis
1	Top 1: real, grid of passport-style faces (AI prob 0.98)	A collage of many perfectly lit, evenly framed faces looks synthetic. The same background and lighting is typical for generated faces.
2	Top 2: real, girl with head wrap (0.97)	Professional portrait with blurred background and very smooth skin. This is the look of AI portraits.
3	Top 3: AI, green and orange mouths (0.03)	A flat graphic poster. It has no photographic texture, so the model cannot find the usual AI signs and calls it real.
4	Top 4: real, Mona Lisa and Arcimboldo (0.96)	These are paintings. Strong painted style and odd faces look like AI art. The Real class contains classical art.
5	Top 5: real, four women in traditional clothes (0.94)	Very saturated colours and a lot of detail, similar to AI generated images of people.
6	Bottom 1: real, paint splashing from a jar (0.92)	It looks like a 3D render with glossy colours. Probably digital art, so it is close to the AI class (possible label noise).
7	Bottom 2: real, black and white portraits (0.91)	High contrast monochrome faces with smooth gradients. Resizing to 160 pixels also removes the film grain.
8	Bottom 3: real, the face grid again (0.91)	The same kind of collage as image 1 (a similar file), so the same reason.
9	Bottom 4: AI, desert and tree at sunset (0.10)	A simple landscape with smooth gradients and little detail. Looks like a real sunset photo.
10	Bottom 5: real, smiling group of friends (0.88)	A bright stock photo with heavy editing and perfect smiles, which resembles generated stock images.

The 10 files and their probabilities are in `reports/misclassified.csv`. The main pattern is that most mistakes are real images that are very polished (studio portraits, collages, paintings). The model probably learned that clean, smooth, high quality means AI. These hypotheses are guesses from looking at the pictures; I did not test them one by one.

2.7 Tool lab: Keras Tuner

Toy part. In `toyrun/keras_toy.py` I used Keras Tuner with random search (8 trials) on Fashion-MNIST, searching the number of units and the learning rate. Figure 24 shows the trial summaries and the table of all trials.

Trial	Units	Learning rate	Validation accuracy
4	128	0.00031	0.8310
3	160	0.00014	0.8263
7	256	0.00012	0.8232
6	256	0.00015	0.8230
0	192	0.00012	0.8207
5	256	0.00172	0.8200
2	256	0.00051	0.8183
1	128	0.00035	0.8130



Figure 24. *Keras Tuner toy run on Fashion-MNIST: best trials and the table of all 8 trials. Best: 128 units, learning rate 0.00031.*

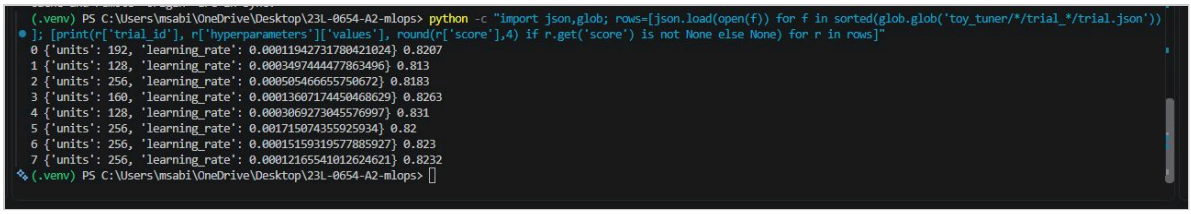


Figure 25. *Reading the trial.json files of the tuner folder (trial id, hyperparameters and score).*

Apply part. `tune.py` runs random search for VisionGuard with 10 trials. It searches the learning rate, dropout and the number of unfrozen layers of MobileNetV2 and minimises validation loss. Results from `reports/tuner_results.csv`:

Trial	Learning rate	Dropout	Unfrozen layers	Validation loss
01	3e-05	0.5	20	0.549
03	3e-05	0.5	40	0.588
07	8e-05	0.3	20	0.595
02	1.2e-04	0.2	20	0.613
09	2.3e-04	0.5	20	0.626
00	1.9e-04	0.2	20	0.639
05	1e-05	0.4	0	0.685
08	1e-05	0.2	40	0.712
06	4e-05	0.3	60	0.841
04	6.7e-04	0.2	60	1.075

Discussion. The best trial used a small learning rate (3e-5), dropout 0.5 and 20 unfrozen layers. The two worst trials both had 60 unfrozen layers, and the worst also had the biggest learning rate (6.7e-4). So the **learning rate together with the number of unfrozen layers mattered most**: opening many layers with a big step damages the pretrained weights. Dropout looks less important (the trials with 0.5 are at the top but they also have small learning rates, so I cannot separate the effects with 10 trials). Choosing the trial with the best validation score can overfit the validation set: with only 146 validation images the score has noise, and when I pick the best of many trials I partly pick the one that was lucky on those images. Its score will usually look a little better than what I will get on new data. That is why I keep the test set for the end.

Bonus (Grad-CAM): I did not do the optional bonus.

Task 3: Experiment tracking with MLflow and DagsHub

3.1 Tool lab: MLflow locally

What I did. `toyrun/toy_local.py` trains a tiny Keras network (64 units, 5 epochs) on a built-in dataset and logs the parameters (`lr`, `epochs`, `units`), the per-epoch metrics, one plot (the loss curve) and the model to a local MLflow store. I ran it twice, with learning rate 0.01 and 0.001 (Figure 26), and then started the MLflow interface with `python -m mlflow ui --backend-store-uri sqlite:///mlflow.db` and compared the two runs (Figure 27).



Figure 26. *Two runs of the toy script with learning rates 0.01 and 0.001, and starting the local MLflow UI.*

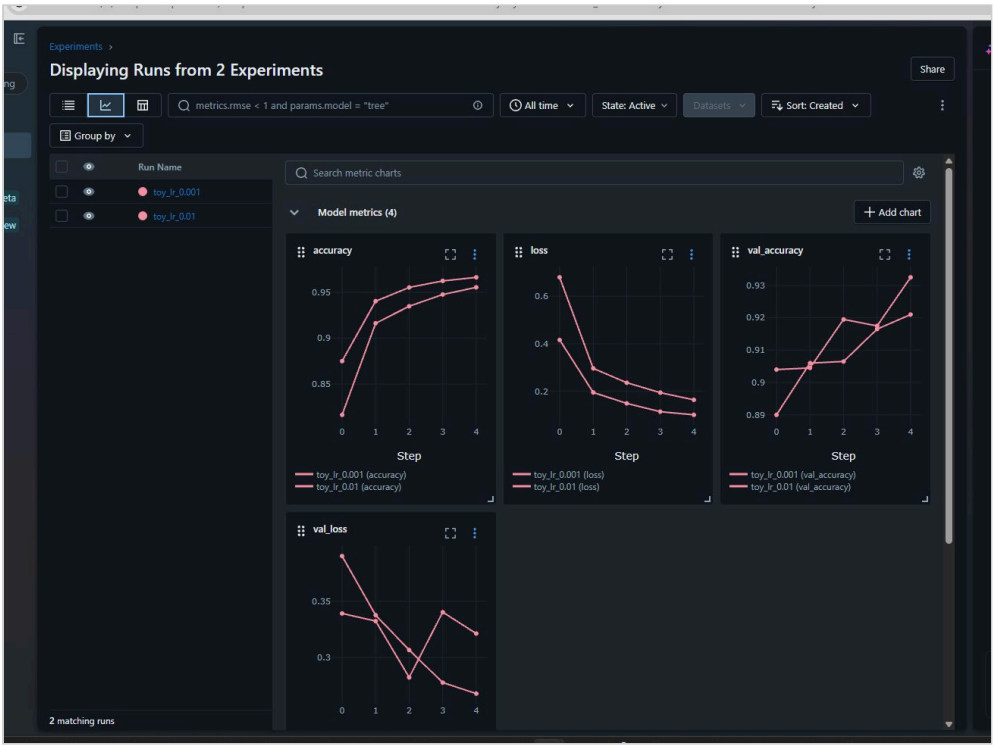


Figure 27. *MLflow comparison of the two toy runs (toy_lr_0.001 and toy_lr_0.01): accuracy, loss, validation accuracy and validation loss per epoch.*

Comparison. After 5 epochs, lr 0.01 reached a validation accuracy of 0.9325 and lr 0.001 reached 0.921. The larger learning rate learns faster on the training set, but its validation loss (0.32) is worse than for 0.001 (0.268) and it is not stable (the validation loss goes up and down), so the smaller one is the safer choice.

Definitions in my own words

- **Experiment**: a folder or group for one problem. All the tries to solve the same task are collected there (mine is called *visionguard*).
- **Run**: one execution of the training script, with its own settings, results and files.

- **Parameter**: an input setting I choose before training and that does not change during the run, like learning rate or dropout.
- **Metric**: a number that measures how well the run went, like accuracy or loss. It can be logged at every epoch and drawn as a curve.
- **Artifact**: a file that a run produces and I want to keep, like a plot, the model or the list of packages.
- **Tag**: a free label attached to a run, like the git commit, the data version or the seed. Used for searching and for knowing where a run came from.

What went wrong. MLflow 3.17 refuses the old local folder store (`file:./mlruns`), so I use an SQLite database file (`sqlite:///mlflow.db`) for the local store. Also `mlflow.exe` in the virtual environment did not start (same moved-folder problem as DVC), so I started it with `python -m mlflow`.

3.2 DagsHub setup

What I did. I created the DagsHub project **VisionGuard** and connected it to my GitHub repository (Figure 28 shows the push to GitHub and Figure 29 the DagsHub page that says it is connected to the GitHub repository). On the DagsHub page I took the MLflow tracking address (`https://dagshub.com/MSabihkhan/VisionGuard.mlflow`) and made an access token. They go into environment variables `MLFLOW_TRACKING_URI`, `MLFLOW_TRACKING_USERNAME` and `MLFLOW_TRACKING_PASSWORD` in a `.env` file that the code reads with python-dotenv. The code never contains the token. The file `.env` is in `.gitignore` and `git check-ignore -v .env` proves it (Figure 31); only `.env.example` with empty placeholders is committed. I repeated the toy run against DagsHub (Figure 30).



Figure 28. *Git remote added and the first push to GitHub.*

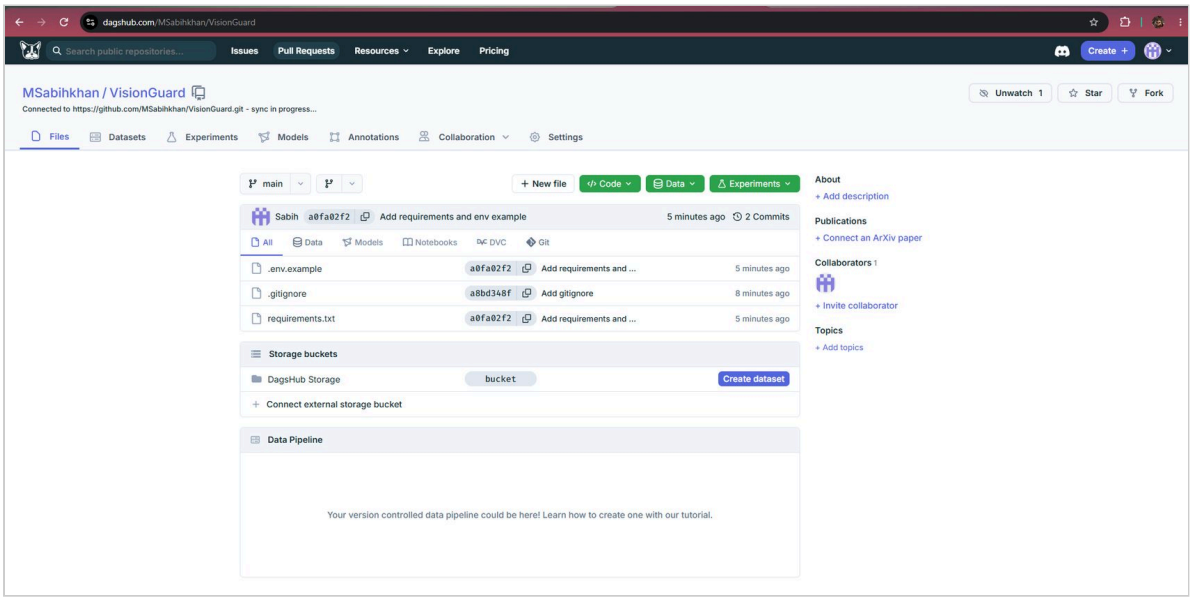


Figure 29. *DagsHub project MSabihkhan/VisionGuard connected to the GitHub repository.*

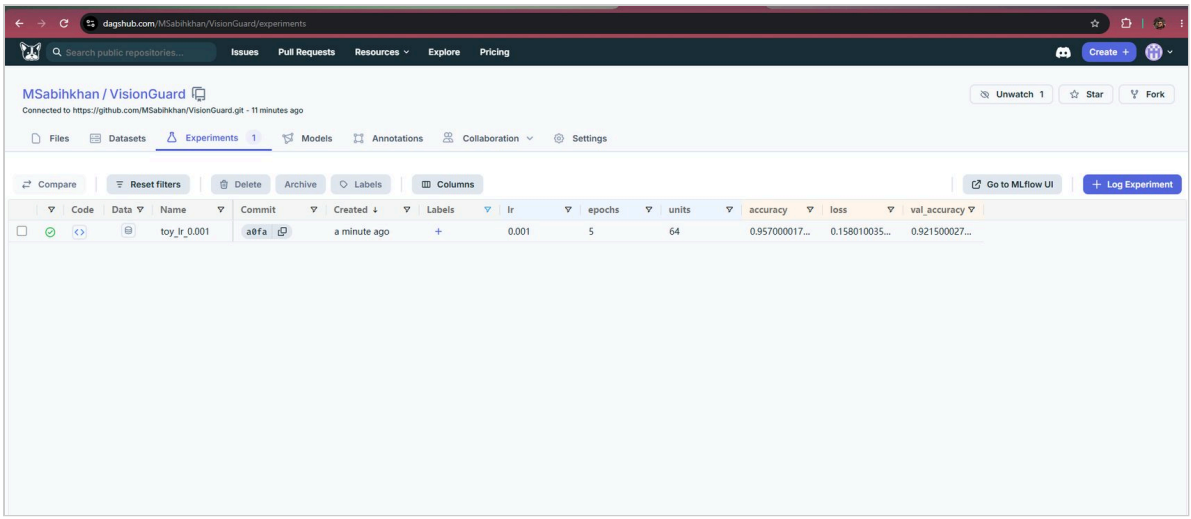


Figure 30. *The toy run toy_lr_0.001 logged to DagsHub (learning rate, epochs, units, accuracy, loss, val_accuracy).*

```
  nothing added to commit but untracked files present (use "git add" to track)
(.venv) PS C:\Users\msabi\OneDrive\Desktop\23L-0654-A2-mlops> git check-ignore -v .env
  .gitignore:1:.env       .env
(.venv) PS C:\Users\msabi\OneDrive\Desktop\23L-0654-A2-mlops> git status
  On branch master
  Untracked files:
    (use "git add <file>..." to include in what will be committed)
          .env.example
          requirements.txt

  nothing added to commit but untracked files present (use "git add" to track)
(.venv) PS C:\Users\msabi\OneDrive\Desktop\23L-0654-A2-mlops>
```

Figure 31. *git check-ignore shows .env is ignored by .gitignore (line 1). Only .env.example is untracked-to-add.*

Why. A token in code or in git history can be read by anyone with access to the repository and is very hard to remove again. With environment variables the secret stays on my machine.

3.3 Instrument VisionGuard training

What I did. `train_tracked.py` trains the transfer-learning model (frozen backbone with a small head) and logs everything to the experiment *visionguard*. Options on the command line change one thing at a time: `--lr`, `--dropout`, `--backbone`, `--no-augment`, `--unfreeze`, `--img` and `--approach`. Logged:

- **parameters**: backbone, image size, learning rate, dropout, unfrozen layers, augmentation, max epochs, batch size, seed and the chosen threshold;
- **per-epoch metrics**: loss, accuracy, val_loss, val_accuracy and learning rate;
- **final metrics**: best epoch, best validation loss, validation F1, validation ROC-AUC and test accuracy, precision, recall, F1 and ROC-AUC;
- **tags**: `git_commit` (from `git rev-parse HEAD`), `dvc_data_version` (the hash in `data/raw.dvc`), `seed` and `approach`.

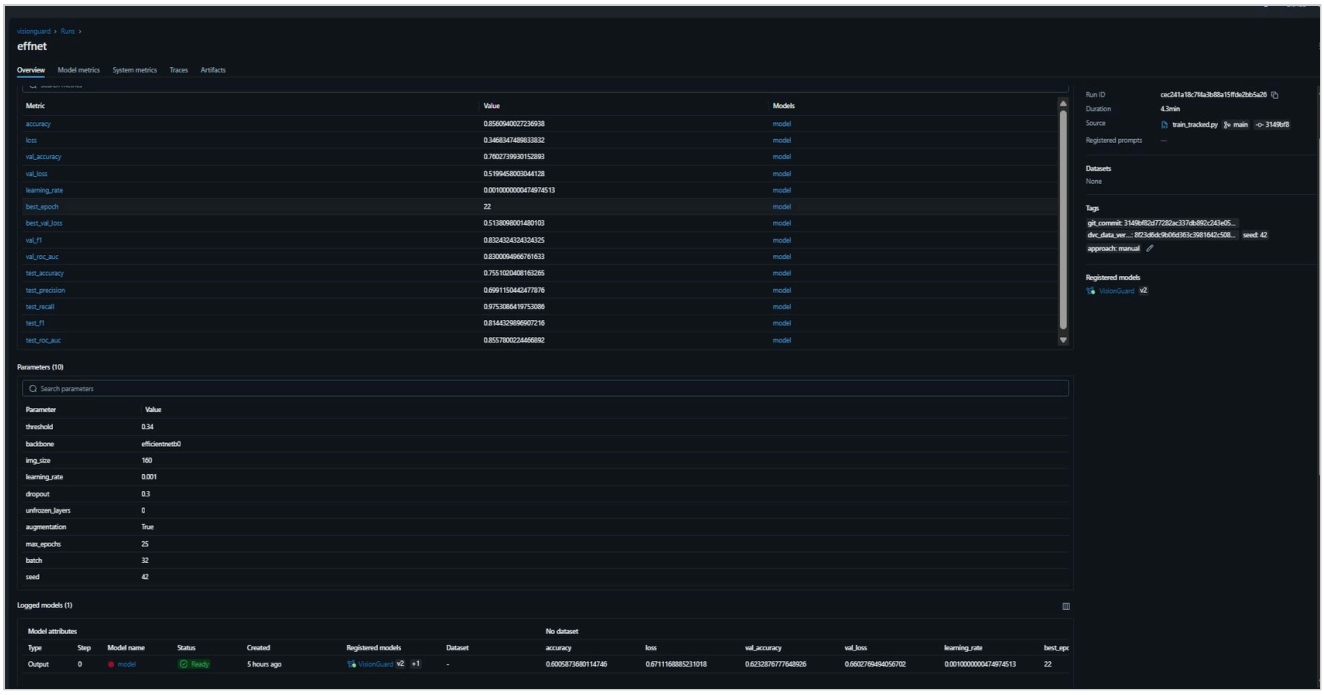


Figure 32. *Run page of the effnet run: metrics, the 10 parameters and the tags (git_commit, dvc_data_version, seed 42, approach manual). The run is linked to registered model VisionGuard v2.*



Figure 33. *Model metrics tab of the effnet run: per-epoch accuracy, loss, val_accuracy and the final metric bars.*

Manual vs autologging. I ran the training once with a Keras callback that calls `mlflow.log_metrics` at the end of every epoch (**manual**, the run *base_manual* and all other runs) and once with `mlflow.tensorflow.autolog()` (**autolog**, the run *base_autolog*). Manual logging only records what I decide to put in, but I know exactly what is in it, and I can add things such as the best epoch, the threshold, the F1 and the tags. Autologging needs no code, and it also captured the model summary, the training parameters (for example the batch size and the optimizer) and a TensorBoard log folder and checkpoints that I did not ask for. But it only knows about what Keras does: it does not know about the test set, the threshold or the git and data tags, so I still log those by hand in both cases. The two runs end with almost the same scores (validation F1 0.798 manual and 0.802 autolog), as expected because they are the same model.

What went wrong. In autolog mode MLflow already logs a parameter called `batch_size`. My script also logged `batch_size` with the same value type, and MLflow stopped with *Changing param values is not allowed*. I renamed

my own parameter to `batch`.

3.4 Artifacts

What I did. At the end of each run the script saves and attaches: the confusion matrix and the ROC curve of the test set (PNG), the three split files `splits/train.csv`, `val.csv` and `test.csv`, the package list (`pip_freeze.txt`) and the trained Keras model (`mlflow.keras.log_model`). Anyone can then see which images were used and with which package versions the model was made.



Figure 34. *Artifacts tab of the effnet run: splits folder, confusion_matrix.png, pip_freeze.txt and roc_curve.png. The confusion matrix is open (32 real correct, 34 real called AI, 2 AI called real, 79 AI correct).*

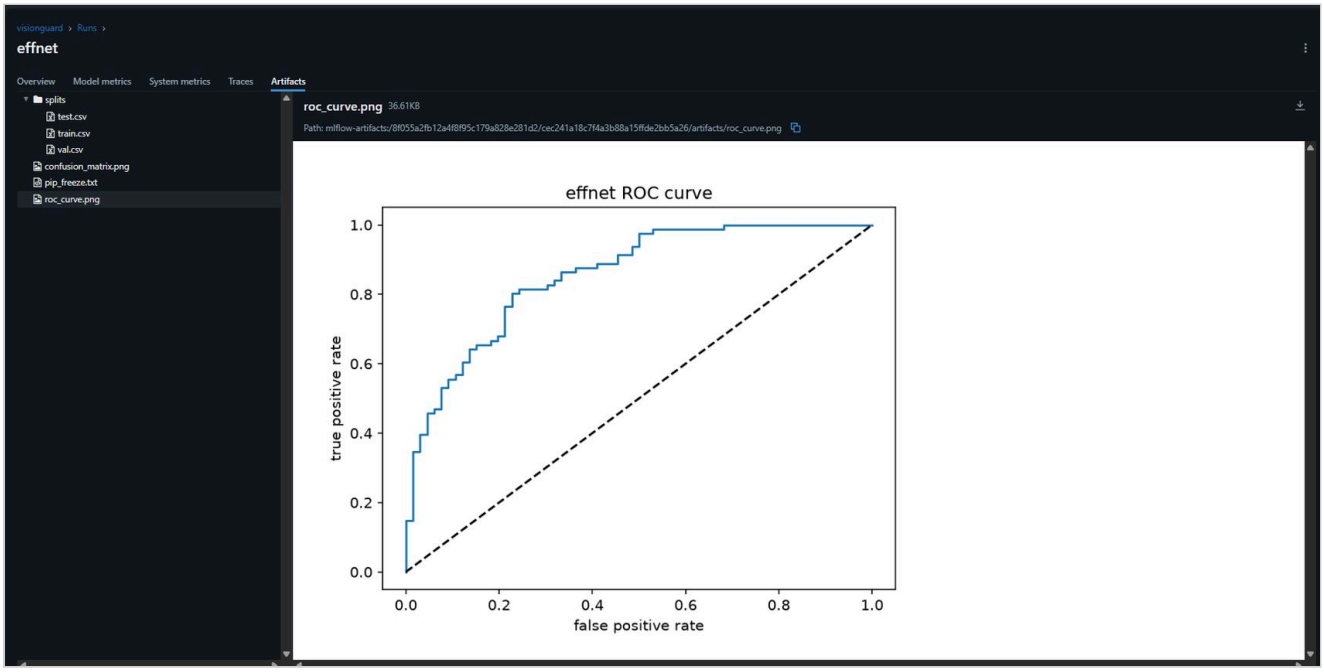


Figure 35. *Artifacts tab of the effnet run with the ROC curve open.*

The trained model is attached to the run as a logged model (it appears in the models view, Figure 38) and not as a plain file in the artifacts list, because MLflow 3 stores models separately.

3.5 Experiment comparison

What I did. I made 6 runs. Each one differs from the base run *base_manual* (MobileNetV2, lr 1e-3, dropout 0.3, augmentation on) by one change:

Run	Change from base	Best epoch	Threshold	Val F1	Test accuracy	Test recall	Test F1	Test ROC-AUC
base_manual	none (reference)	15	0.44	0.798	0.741	0.827	0.779	0.790
base_autolog	logging with autolog (same model)	25	0.31	0.802	0.728	0.926	0.789	0.802
lr_low	learning rate 1e-4	25	0.18	0.741	0.578	0.975	0.718	0.645
drop_high	dropout 0.6	23	0.38	0.793	0.741	0.877	0.789	0.798
no_aug	no augmentation	16	0.39	0.814	0.748	0.864	0.791	0.805
effnet	backbone EfficientNetB0	22	0.34	**0.832**	**0.755**	0.975	**0.814**	**0.856**

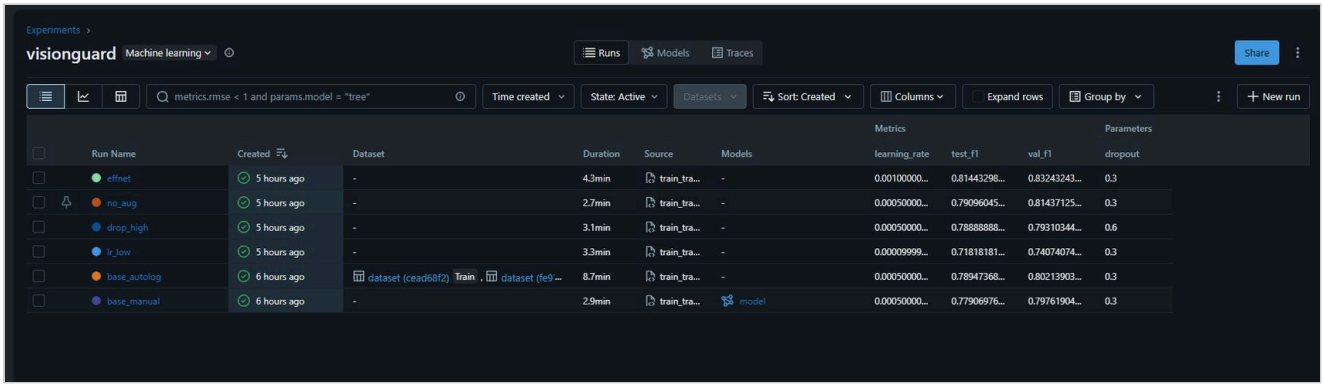


Figure 36. *The visionguard experiment on DagsHub with the six runs (learning rate, test F1, validation F1 and dropout columns).*

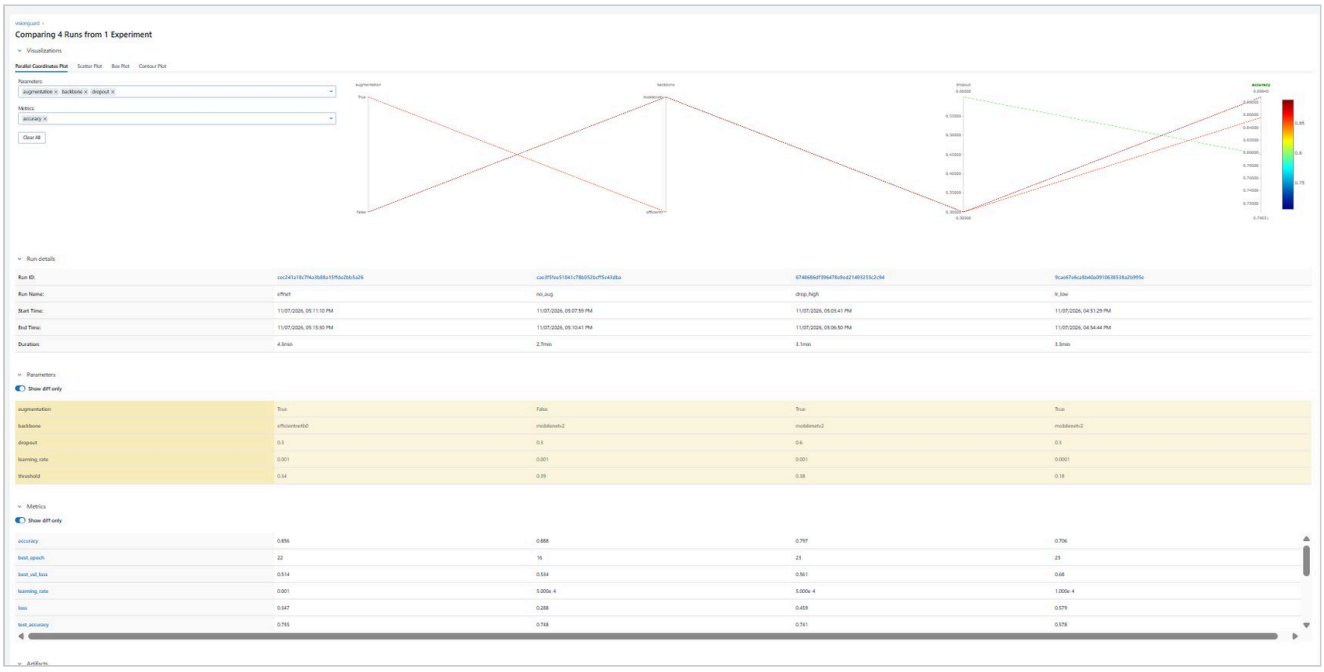


Figure 37. *Compare view of four runs (effnet, no_aug, drop_high, lr_low): parallel coordinates, differing parameters and metrics.*

Choice and justification. I chose **effnet**. I compare on the **validation F1** because the threshold and all decisions are made on validation data, and F1 is better than accuracy for a dataset that is not balanced and where both kinds of error matter. EfficientNetB0 has the highest validation F1 (0.832) and also the lowest validation loss (0.514), and the test results agree (ROC-AUC 0.856, the best). The test numbers are shown only to report the final result and were not used to choose. I also looked at the ROC-AUC because it does not depend on the threshold.

Things to be careful about. The test set has 147 images and each run used one seed, so differences of one or two points (for example between base_manual and drop_high) are inside the noise and I do not claim that dropout 0.6 is better or worse. Only two changes look clearly meaningful: a too small learning rate (lr_low) is clearly worse, and the bigger EfficientNetB0 is better. I did not run different image sizes, and the number of unfrozen layers was explored in the Keras Tuner (2.7) instead. In effnet the recall is 0.975 but 34 real images are still called AI, which is the weak point I discussed in 2.5.

3.6 Model registry

What I did. `register_model.py` searches the finished runs, sorts them by validation F1 and registers the best two as two versions of the model **VisionGuard**: version 1 from *no_aug* (F1 0.814) and version 2 from *effnet* (F1 0.832). Version 2 got the alias **champion**. `load_champion.py` is a small separate script that loads `models:/VisionGuard@champion` and classifies one image. `download_champion.py` downloads the champion file once to `models/champion/model.keras` and writes `info.json` with the version and the threshold (0.34).

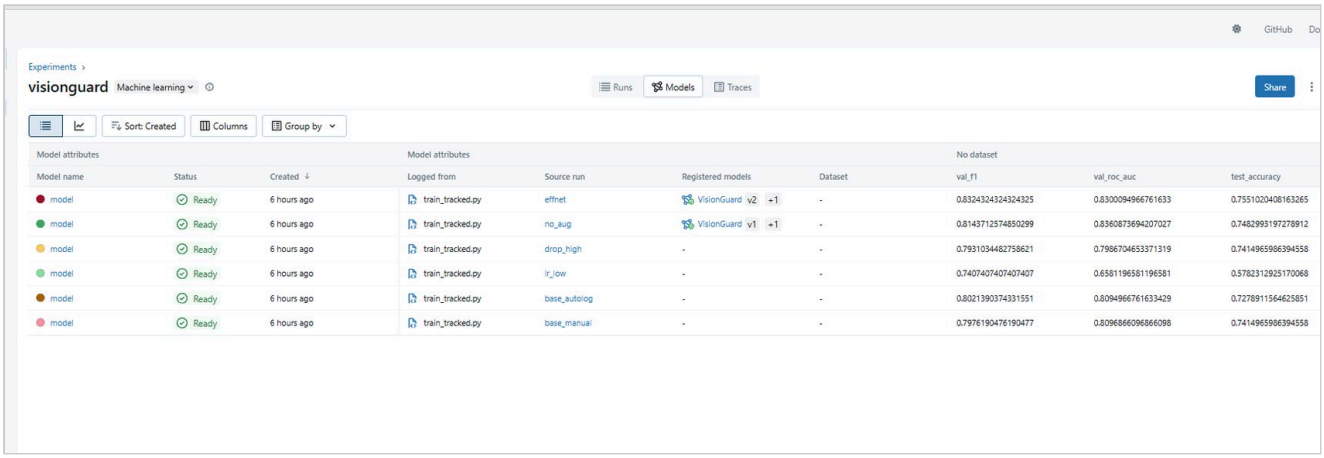


Figure 38. *Models view of the experiment: effnet is registered as VisionGuard v2 and no_aug as VisionGuard v1.*

How the model file reaches the server. The web app must not call DagsHub for every request: it would be slow, would need the token on the server and would break if DagsHub is down. So I run `download_champion.py` once (before building or deploying). The server then only reads the local file `models/champion/model.keras` at start-up. In Docker Compose the folder is mounted into the container, and for the hosted API the file is built into the image (see Tasks 6 and 7). To change the champion I move the alias, download again and restart.

What went wrong. The first time I ran `register_model.py` it created the registered model and then failed with *Unable to find a logged_model with artifact_path model under run ...*. MLflow 3 stores a model as its own object, not under the run's artifact folder, so the old `runs:/<run_id>/model` address does not work on DagsHub. I changed the script to look up the logged model that belongs to each run and register it with `models:/<model_id>`. After that both versions registered.

Task 4: Flask web application

4.1 Inference module

What I did. `inference.py` is separate from the Flask code. When it is imported it reads the settings (`MODEL_PATH`, `THRESHOLD`), reads `info.json` beside the model for the version and threshold, and loads the Keras model **once**. The function `predict(raw_bytes)` prepares the image with `preprocess.prepare()`, runs the model and returns the label (AI-generated or Real) and a confidence. If the probability is at least the threshold the label is AI-generated and the confidence is that probability; otherwise it is Real with confidence 1 minus the probability. A file that cannot be decoded raises `ValueError`.

Why load only once. Loading a Keras model takes several seconds and a lot of memory. If it were loaded for each request every user would wait seconds, and several parallel requests could run out of memory. At start-up it is done a single time and each request only runs a fast forward pass.

Why preprocessing must match training. The model learned from images that were decoded to 3 channels and resized to 160 x 160 with `tf.image.resize` and then scaled inside the model. If the app used another library or another resize method, the pixel values would be slightly different from what the model learned on (training-serving skew), and the accuracy would drop without any error message. So training (`data.py`) and serving (`inference.py`) both call the same function `prepare()` in `preprocess.py`, and a test checks that it keeps the pixel range.

4.2 Routes and validation

What I did. `app.py` has four pages: `/` (upload form), `/predict` (POST, shows the result), `/history` (table from the database) and `/health` (JSON). Checks before predicting:

- **wrong file type**: only .png, .jpg and .jpeg are allowed (checked on the sanitized name);
- **too large**: `MAX_CONTENT_LENGTH` is 5 MB (set by `MAX_UPLOAD_MB`); Flask answers with error 413 and my own error page;
- **looks like an image but cannot be decoded**: the decode raises an error, I catch it and show a message;
- **file names** go through `werkzeug.utils.secure_filename`, so something like `../../etc/passwd.png` becomes a harmless name;
- the upload is read into memory and shown again as a base64 data address, so **no image is ever written to disk**;
- friendly error pages for 404, 413 and 500 (`error.html`).

The automated tests for these cases are in `tests/test_app.py` (wrong type, undecodable image, too large, history with a dirty file name) and all pass (Figure 41).

4.3 HTML and CSS

What I did. I wrote the templates in `templates/` (a base layout, the upload page, the result page, the history page and the error page) and one stylesheet `static/style.css`. The upload page has a file picker (several files are allowed, used in Task 8) and a button. The result page shows the uploaded image, the verdict in colour, the confidence as a percentage and a bar, the model name, and a yellow note saying that this is a statistical estimate and not proof. There is a menu link to the history table (Figure 39 and Figure 42).

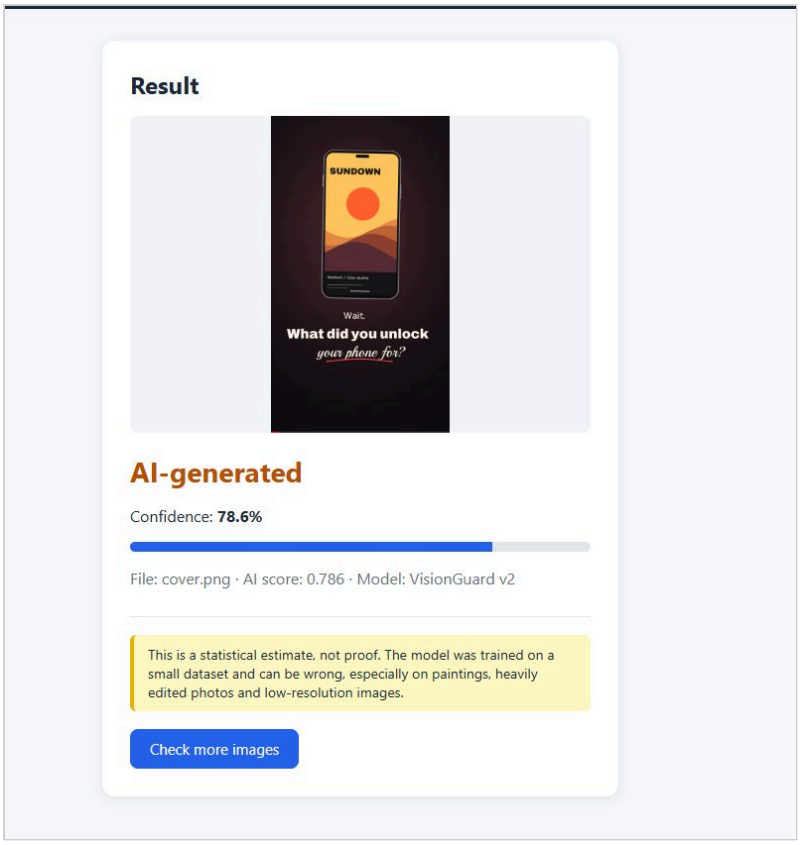


Figure 39. *Result page for an uploaded image: verdict, confidence bar, AI score, model version and the honest caveat.*

About the confidence. The confidence is the probability of the *chosen* class. Because the threshold is low (0.34), an image with an AI score of 0.40 is called AI-generated but shows a confidence of only 40%. This can look strange, but it is honest about how sure the model is.

4.4 Configuration and production server

What I did. All settings come from environment variables (read from `.env` or from the container): `MODEL_PATH`, `THRESHOLD`, `DATABASE_URL`, `MAX_UPLOAD_MB`, `HOST`, `PORT`, and for the hosted API `API_URL`, `API_KEY`. A `.env.example` lists them without values. The app is started with `app.run(..., debug=False)`.

Why debug is off. Flask debug mode shows an interactive page in the browser where anyone can run Python code if they get to an error, and it restarts the server when files change. This is safe on my laptop but a serious hole on a public server, so it is always off here.

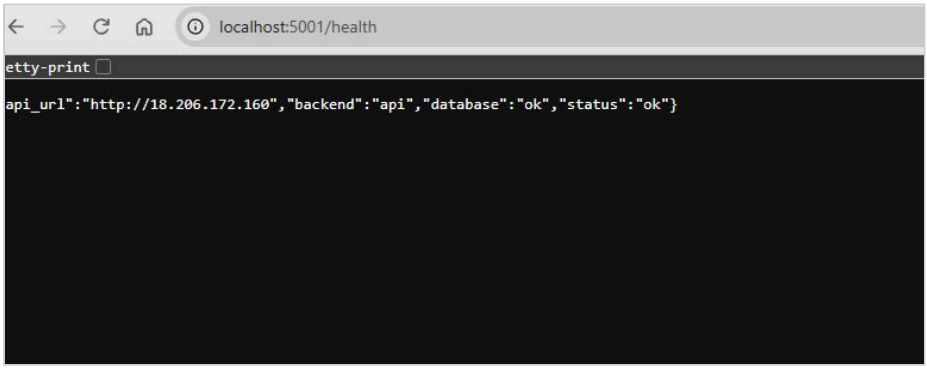


Figure 40. *The /health endpoint of the Flask app on localhost:5001. It shows the database is ok and which backend is used (here the hosted API).*

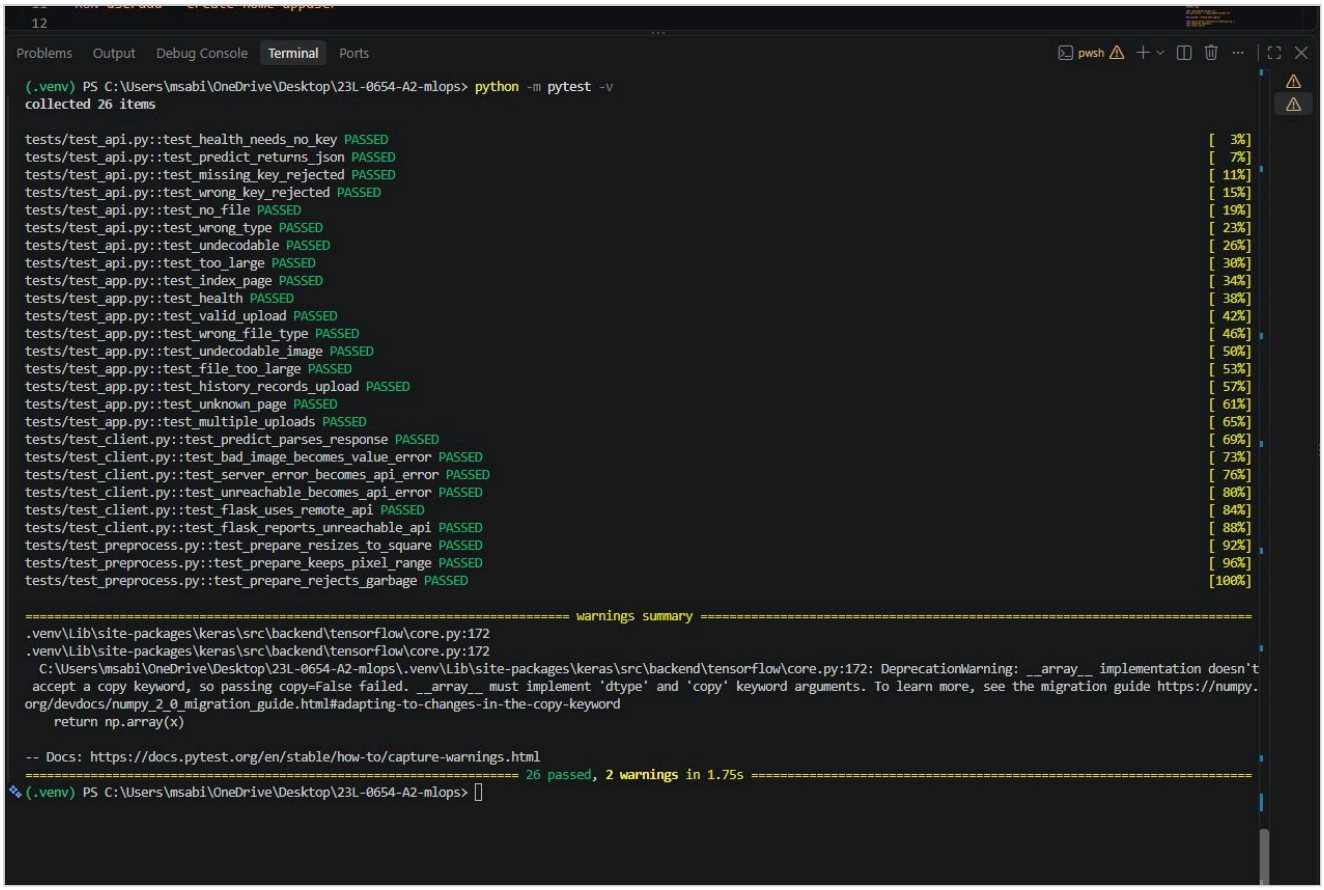


Figure 41. *pytest -v: all 26 tests pass (API, Flask app, client and preprocessing).*

Task 5: Database trail

What I did. `db.py` uses SQLAlchemy. The table `predictions` has the columns `id`, `created_at` (UTC time), `filename`, `label`, `confidence`, `ai_score` and `model_name`. After each successful prediction, the Flask view calls `add_prediction` and a row is written. This answers the four questions of the assignment: **what** was decided (label), **when** (created_at), **by which model** (model_name, for example VisionGuard v2) and **how confidently** (confidence and ai_score). The `/history` page shows the latest rows, newest first (Figure 42).

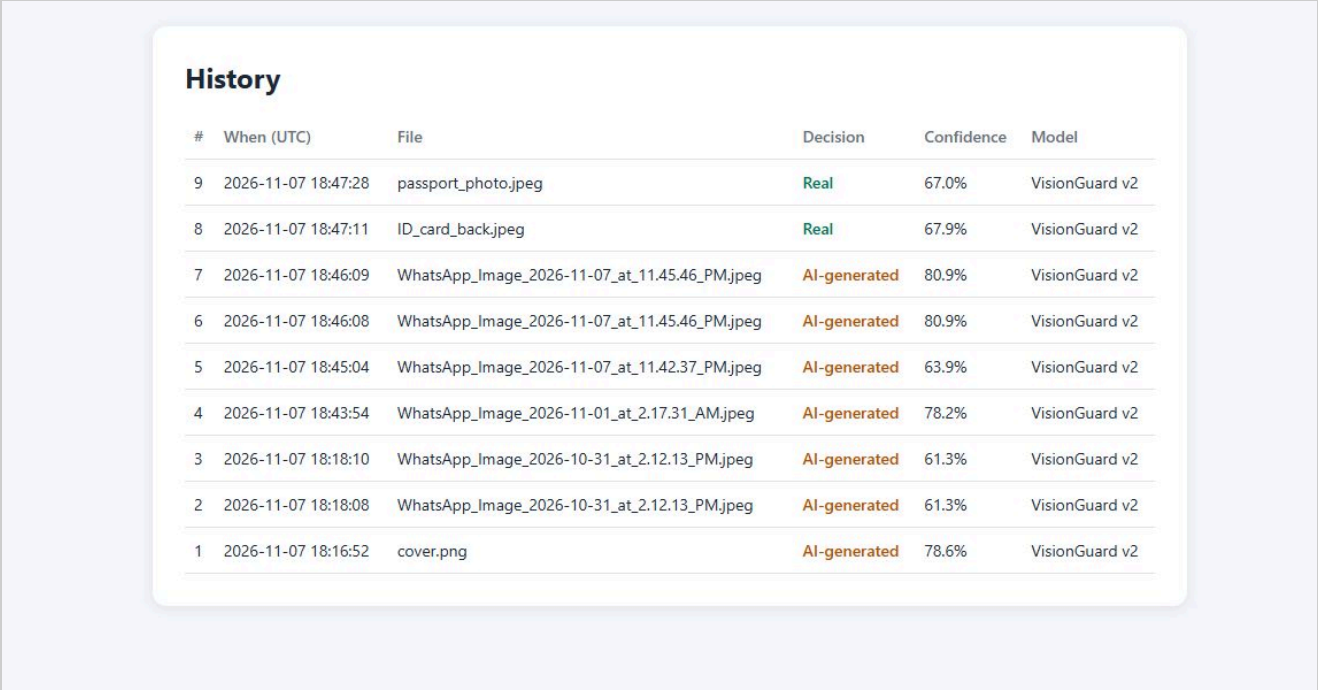


Figure 42. *History page: nine requests with time (UTC), file, decision, confidence and model version.*

Which database and why. The code reads `DATABASE_URL`. On my laptop it is an SQLite file (`sqlite:///history.db`), which needs no server, and in Docker Compose it is PostgreSQL (see Task 6). The same code works for both because SQLAlchemy hides the differences. Only filenames and results are stored, never the images, which also protects privacy.

Design note. A row is written only after a successful prediction, so rejected files (wrong type, too large, not decodable) do not appear in the history. The filename stored is the sanitized one.

Task 6: Containerization with Docker and Docker Compose

6.1 Docker warm-up

What I did. With the ready-made `nginx` image I ran a container with a mapped port (`docker run -d --name toy -p 8080:80 nginx`), read its logs (`docker logs toy`), opened a shell inside it (`docker exec -it toy sh`, then `ls`), made a folder with a small HTML page and mounted it as a volume (`-v ...:/usr/share/nginx/html`), edited the page on my computer and saw the change in the browser without restarting, listed containers and images (`docker ps`, `docker ps -a`, `docker images`), stopped and removed containers (`docker stop`, `docker rm`) and removed the image (`docker rmi nginx`). I also made a named volume (`docker volume create demo_data`, `docker volume ls`) and looked at the layers of the image with `docker history`.



Figure 43. *Terminal: docker version, run, logs, exec sh, mkdir and a volume mount, ps, ps -a, images, stop, rm and rmi.*

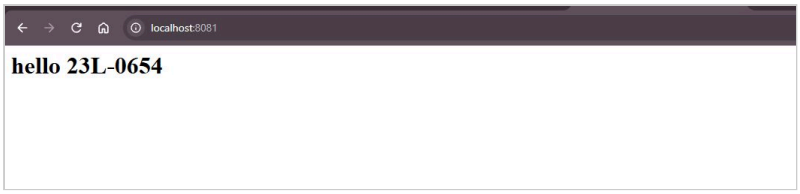


Figure 44. *The nginx container serving my mounted page on localhost:8081 (hello 23L-0654).*



Figure 45. *After editing index.html on my computer, the same container shows the new text without restart (the bind mount works).*



Figure 46. *A second container with a mounted site folder (version 1).*



Figure 47. *The page after I changed the file on the host (version 2).*

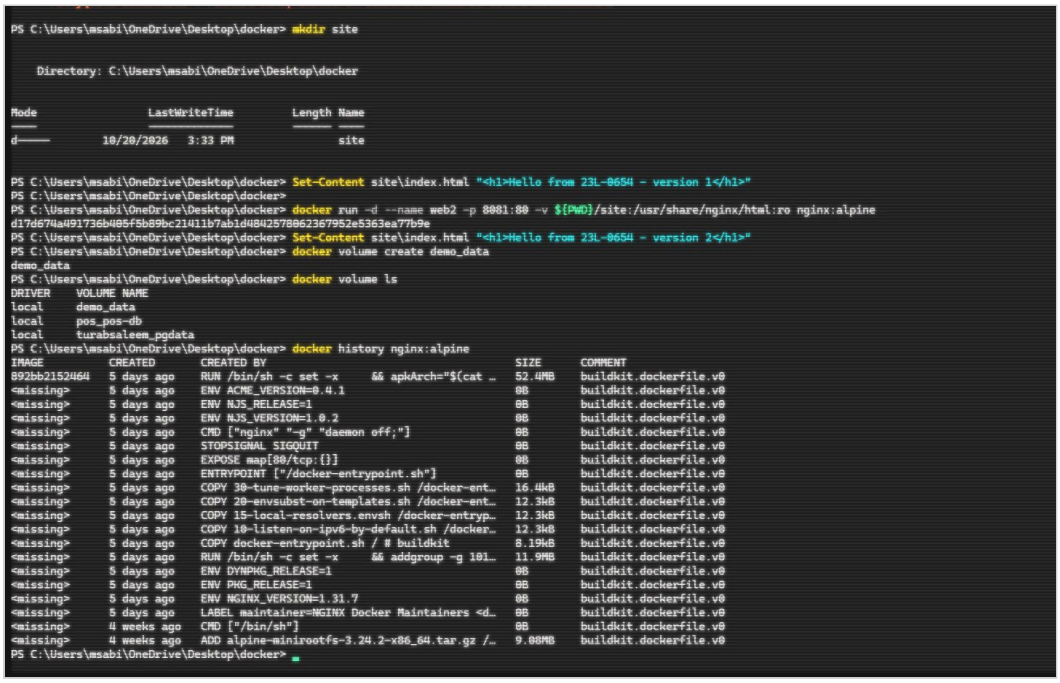


Figure 48. *docker volume create / docker volume ls and docker history nginx:alpine (the layers of the image).*

Definitions in my own words

- **Image**: a read-only package that holds the program, its libraries and settings; the recipe to start a container.
- **Container**: a running (or stopped) instance of an image, isolated from my computer, with its own files and processes.
- **Layer**: every instruction in a Dockerfile makes a layer of file changes. Layers are cached and shared, so if an instruction and everything before it did not change, Docker reuses it.
- **Volume**: storage that lives outside the container, so data (for example a database) survives when the container is removed.
- **Port mapping**: connects a port on my computer to a port inside the container (`-p 8080:80` means localhost:8080 goes to port 80 of the container).

6.2 Dockerfile for the web app

What I did. The `Dockerfile` in the repository does this in order:

```
FROM python:3.13-slim
ENV PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1 TF_CPP_MIN_LOG_LEVEL=2
WORKDIR /app
COPY requirements-docker.txt .
RUN pip install -r requirements-docker.txt
RUN useradd --create-home appuser
COPY app.py client.py db.py inference.py preprocess.py ./
COPY templates templates
COPY static static
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s --start-period=60s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:5000/health')"
CMD ["python", "app.py"]
```

- **Order and layer cache.** `requirements-docker.txt` is copied and installed *before* the source code. Installing TensorFlow takes minutes and about 1.7 GB, and the requirements almost never change, while my source changes all the time. Because of the cache, after editing `app.py` Docker only redoes the last few cheap COPY layers and not the pip install.
- **Slim base:** `python:3.13-slim` is Debian without extra tools (the base layer is only 88 MB).
- **Non-root user:** `appuser` runs the app, so if someone breaks into the app they are not root inside the container.
- **Health check:** Docker calls `/health` every 30 seconds; the status becomes *healthy* or *unhealthy*, and Compose can wait for it.
- **Ignore file:** `.dockerignore` keeps `.env`, `.git`, `data`, `models`, `reports`, `Screenshots` and the virtual environment out of the build, so the image is small and has no secrets.
- **No secrets in the image:** the database password and other settings are given at run time by Compose from the untracked `.env`.

TensorFlow package. I use `tensorflow-cpu==2.22.0rc0` (the same version I trained with, so the saved models load without problems). The normal `tensorflow` package for Linux brings the NVIDIA CUDA libraries (several extra GB) which are useless because the server has no GPU. CPU-only is enough for predicting one image at a time. This is a release candidate, which is not ideal, but I wanted the same version that trained the models.

Link to the Dockerfile: https://github.com/MSabihkhan/VisionGuard/blob/main/Dockerfile

6.3 Compose stack

What I did. `docker-compose.yml` defines two services:

- **db**: `postgres:16-alpine` with a **named volume** `pgdata` so the data survives container removal, and a health check with `pg_isready`. It has **no ports** section, so nothing is published to the outside.

- **web**: built from the Dockerfile, with `depends_on: db: condition: service_healthy`, so the web app starts only when Postgres is really ready. It reaches the database with the service name `db` on the internal Compose network (`postgresql+psycopg2://...@db:5432/...`). Only port 5000 of the web app is published.

The passwords (`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`) come from the untracked `.env` and the file has no default password (`${POSTGRES_PASSWORD:?set ...}` stops with an error if it is missing). `.env.example` shows the names with a placeholder. I do not use `env_file` so the MLflow token in the same `.env` does not enter the container.

How the model reaches the container. I **mount** `./models/champion` read-only into `/app/models/champion` instead of building the model into the image. The image stays smaller and independent from the model; when I register a new champion I only download it and restart, without rebuilding the 2 GB image. The disadvantage is that the container needs the folder next to it. For the hosted API in Task 7 I do the opposite and build the model into the image, since there is no folder on the server to mount from.

Link to the Compose file: https://github.com/MSabihkhan/VisionGuard/blob/main/docker-compose.yml

6.4 Image analysis

What I did. I built the image once without the optimization (`Dockerfile.before`, no `PIP_NO_CACHE_DIR`, so pip keeps its download cache in the image) and once with the final Dockerfile, and compared them with `docker image ls` and `docker history`.

	Before	After
Image size	3.03 GB	2.29 GB
Largest layer (pip install of the requirements)	2.09 GB	1.71 GB
Second largest layer (Debian base)	87.7 MB	87.7 MB

Conclusion. The layer that installs the Python packages is by far the largest, because TensorFlow and its dependencies are big. Not keeping the pip cache removed about 0.7 GB (24%). The base image is small in comparison, so changing it would not help much; the way to go lower would be a multi-stage build or less heavy libraries.

6.5 Operations and troubleshooting

What I did. For both services I ran start, stop, restart, logs and a shell:

```
docker compose up -d
docker compose stop            docker compose start
docker compose restart web     docker compose logs web
docker compose exec web sh     docker compose exec db sh
```

Breaking it on purpose. I started the stack with a wrong database password for the web service (`POSTGRES_PASSWORD=wrongpass123 docker compose up -d` while the Postgres volume was initialised with the right one). The web container does not start properly and its log (`docker compose logs web`) ends with the error `password authentication failed for user "visionguard"`. From the log alone I read that: the app reached the database (so the network and the service name are fine), but Postgres refused the login, so the password in the environment is wrong. **Fix:** start again with the right password from `.env` (`docker compose up -d`); the web service comes up healthy and the history page works again.

What went wrong. I chose the wrong password as the break, because deleting the model file did not fail reliably with a mounted folder on Docker Desktop (the container still showed the file).

6.6 Tool lab: GitHub Actions (CI)

Toy part. In a throwaway GitHub repository I made a small `calc.py` with an `add` function, a test in `test_calc.py` and the workflow `.github/workflows/workflow.yml`, which runs on every push (`on: [push]`), uses a **matrix of**

Python 3.11 and 3.12, installs pytest and runs `pytest -q`. I changed `add` to subtract to make the test fail (**red** run) and changed it back (**green** run). The files are in `toyrun/ci_toy`.

Apply part. `.github/workflows/ci.yml` for VisionGuard has two jobs. **test**: checkout, Python 3.13 with the pip cache, install the requirements, `ruff check .` (linting, with rules set in `pyproject.toml`) and `pytest -q` (26 tests, they build a tiny model so no real model or data is needed). **docker** (runs after test passes): `docker build -t visionguard-web .`. The README has the status badge:

```
[![CI](https://github.com/MSabihkhan/VisionGuard/actions/workflows/ci.yml/badge.svg)]
(https://github.com/MSabihkhan/VisionGuard/actions/workflows/ci.yml)
```

Where workflow secrets live. Secrets are stored in the repository settings on GitHub (**Settings - Secrets and variables - Actions**) and the workflow uses them as `${{ secrets.NAME }}`. They must not be written in the workflow file because that file is committed and readable by everyone who can see the repository (and stays in the git history), while a GitHub secret is encrypted and hidden in the logs. My CI needs no secrets, because the tests use a tiny local model and do not call DagsHub.

What went wrong. The first `ruff check` reported lots of style warnings in my older scripts (import order, a variable named `l`). I fixed the real ones and chose the rule set in `pyproject.toml` (errors, unused imports and import sorting) so the check stays useful and not noisy.

Task 7: API deployment

Pathway. C (another cloud: AWS). As the task says, the Flask UI stays on localhost and only the **model API** is on the cloud. I put the model in a container on an EC2 virtual machine and call it from Flask over HTTP. Compose mapping: the *web* service stays local (and so does its database), the model server becomes the cloud container, the model file is built into the image (instead of a mount) and the network between them is the internet.

What I deployed. `api.py` is a small Flask API (served by gunicorn). `GET /health` returns the model name and threshold. `POST /predict` takes an image in the form field `image` and returns JSON with `label`, `confidence`, `ai_score`, `threshold`, `model` and `filename`. It uses the same `inference.py` and `preprocess.py` as the local app. It checks the type, the size and whether the file can be decoded, and writes a log line for each prediction. Every request to `/predict` needs the header **X-API-Key**, otherwise the answer is 401, so that not everyone on the internet can use my server.

Things I researched first. Free tier: the free-tier instances (micro) have only 1 GB of RAM, which is too little for TensorFlow and the model, so I used a `t3.small` (2 GB). It costs about 0.02 USD per hour, so I stop it after testing. Free-tier terms change, so I checked my own billing page. **Size:** the image with TensorFlow-CPU is about 2.3 GB (440 MB compressed in the registry), which is why I do not use a serverless function (Vercel and Lambda size limits). **Firewall:** the instance only opens port 80 in the security group, and has no SSH port; I read logs with AWS Systems Manager instead. **Weakness:** the API uses plain HTTP, so the key travels unencrypted; for a real product I would add HTTPS with a load balancer or a domain and certificate.

Step by step deployment (AWS CLI)

- **1. Build the image** with the model inside: `docker build -f Dockerfile.api -t 231-0654-a2-ops-api:latest .` (`Dockerfile.api` uses `python:3.13-slim`, `tensorflow-cpu`, gunicorn, a non-root user, a health check and copies `models/champion`).
- **2. Create a registry and push:** `aws ecr create-repository --repository-name 231-0654-a2-ops-api`, `aws ecr get-login-password | docker login ...`, `docker tag` and `docker push` (the pushed image is about 440 MB).
- **3. Store the API key** as a SecureString in AWS Systems Manager Parameter Store (`/visionguard/api_key`), so it is not in the image, in git or in the launch script.
- **4. IAM role** `visionguard-api-role` for the instance, with read-only access to ECR and SSM Parameter Store, plus the Systems Manager core policy (to run commands and read logs without SSH), and an instance profile.
- **5. Security group** `visionguard-api-sg`: inbound TCP port 80 from anywhere, nothing else.
- **6. Launch the instance:** `aws ec2 run-instances` with Amazon Linux 2023, `t3.small`, 20 GB disk, IMDSv2 required, and a user-data script that installs Docker, reads the key from SSM, logs in to ECR and runs `docker run -d --restart unless-stopped -p 80:8000 -e API_KEY=... image`.
- **7. Check** from outside with Postman: `GET http://18.206.172.160/health`, then `POST /predict`.
- **8. Read the container logs** with `aws ssm send-command` running `docker logs`.
- **9. Clean up** after the work: terminate the instance, delete the ECR repository and the SSM parameter, so there are no costs.

API URL. http://18.206.172.160 (`/health` and `/predict`). The address is not an Elastic IP, so it changes if the instance is stopped and started.

Postman tests

Tests. I tested with Thunder Client (a Postman-like client in VS Code) from my machine.

Test	Request	Expected	Result
Health	GET /health	200 and model info	200 OK: model VisionGuard v2, threshold 0.34 (Figure 49)
Valid image	POST /predict, form file `image`, header X-API-Key	200 and JSON	200 OK, label AI-generated, confidence 0.8089 (Figure 50)
No key	POST /predict without the header	401	401 Unauthorized: invalid or missing API key (Figure 51)
Wrong file type	POST /predict with a .gitignore file	400	400 Bad Request: only PNG and JPG images are allowed (Figure 52)

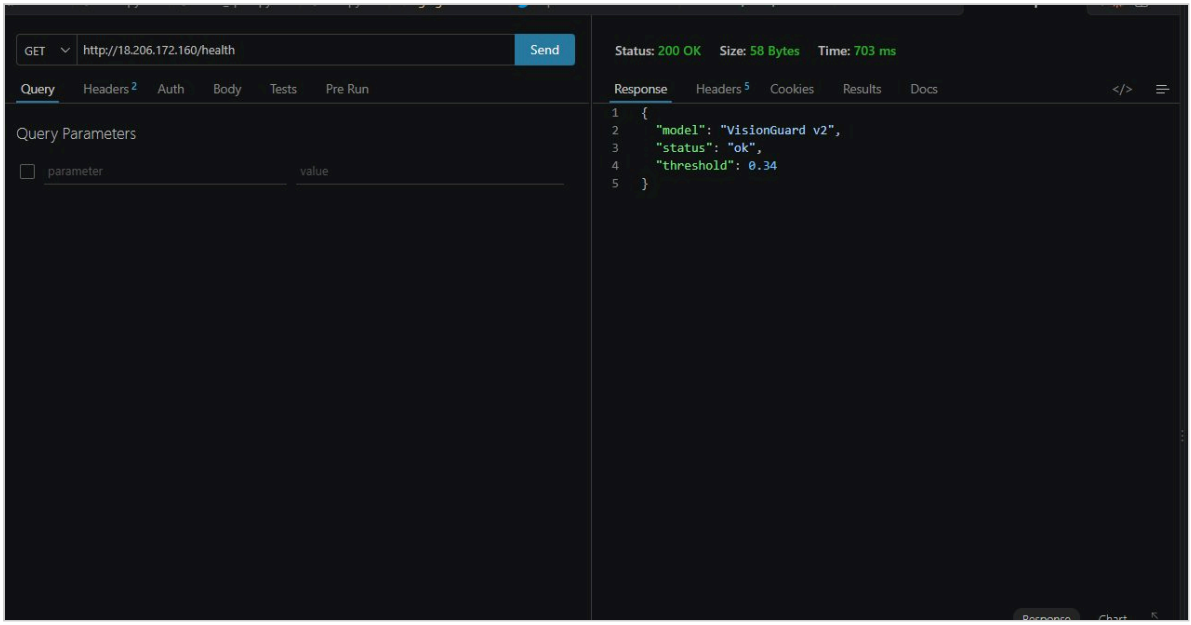


Figure 49. *Postman (Thunder Client): GET /health on the AWS server returns 200 OK.*

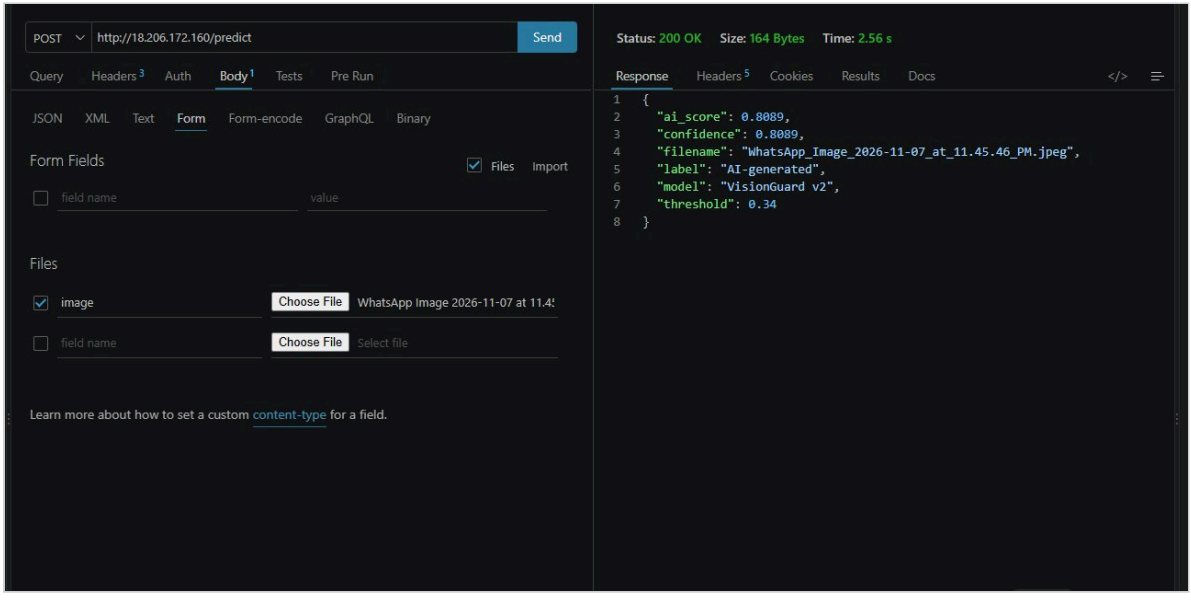


Figure 50. *POST /predict with an image and the API key: 200 OK and the JSON result.*

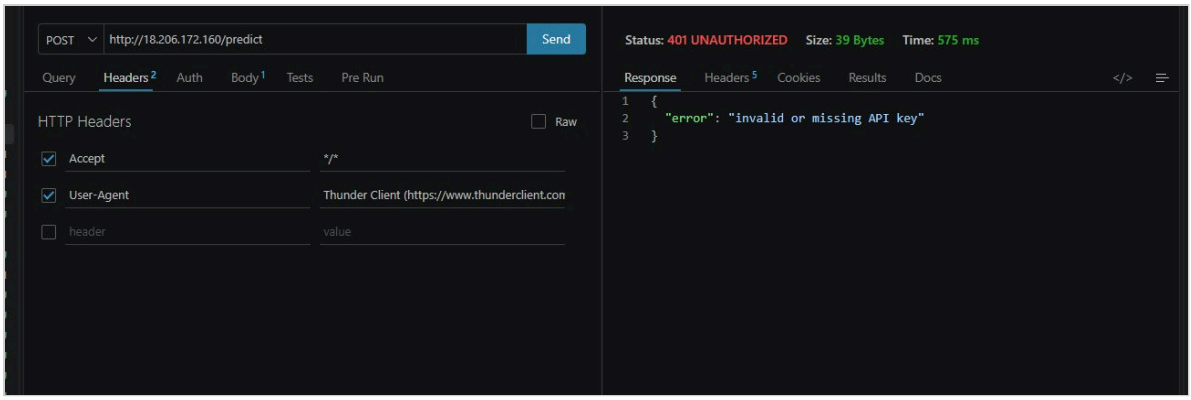


Figure 51. *POST /predict without the API key: 401 Unauthorized.*

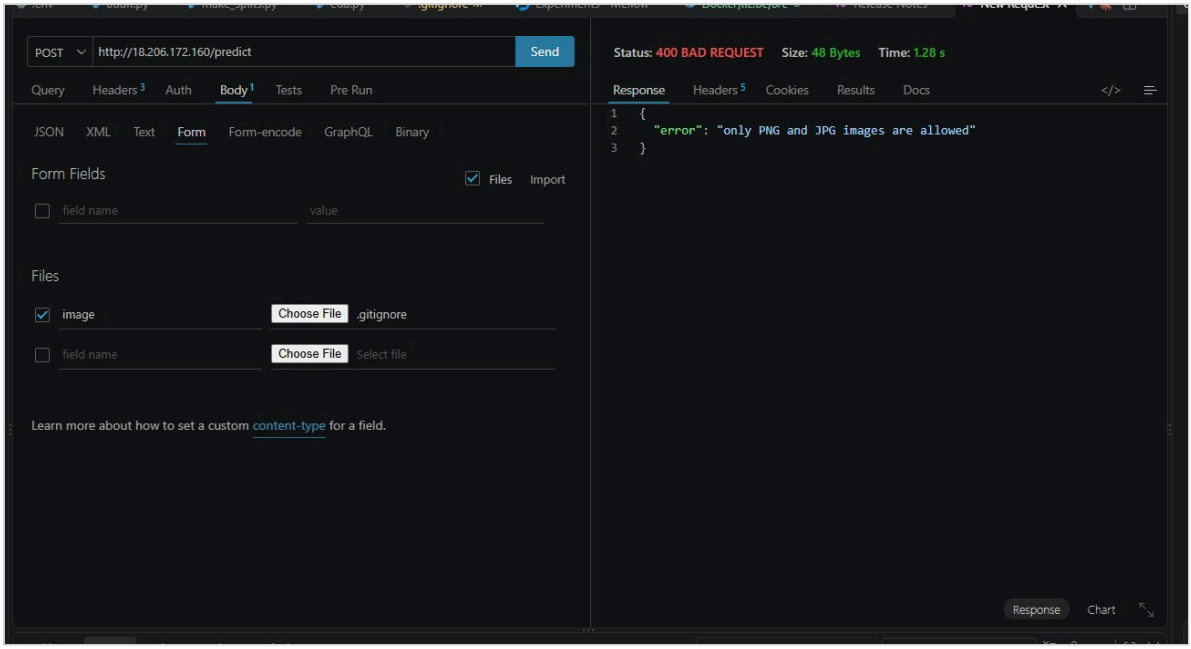


Figure 52. *POST /predict with a file that is not an image (.gitignore): 400 Bad Request.*

Logged result on the server

This is the log of the container on the server (read with `aws ssm send-command` and `docker logs`, times are UTC). It shows my Postman requests, the model decision and the rejected request. The 127.0.0.1 lines every 30 seconds are Docker's own health check.

```
72.255.7.99 - - [07/Nov/2026:18:57:51 +0000] "GET /health HTTP/1.1" 200 58 "-" "Thunder Client"
2026-11-07 18:59:42,320 INFO predict file=WhatsApp_Image_2026-11-07_at_11.45.46_PM.jpeg label=AI-
generated confidence=0.8089 ai_score=0.8089
72.255.7.99 - - [07/Nov/2026:18:59:42 +0000] "POST /predict HTTP/1.1" 200 164 "-" "Thunder Client"
72.255.7.99 - - [07/Nov/2026:19:00:35 +0000] "POST /predict HTTP/1.1" 400 48 "-" "Thunder Client"
2026-11-07 19:01:14,153 WARNING rejected request: bad or missing API key from 72.255.7.99
72.255.7.99 - - [07/Nov/2026:19:01:14 +0000] "POST /predict HTTP/1.1" 401 39 "-" "Thunder Client"
```

What went wrong. Some things I tried first did not work. (1) My first attempt to store the key in AWS Parameter Store was blocked by a permission check, so I generated a new key just for AWS and stored it there, and put the same new key in my local `.env`. (2) The first Docker build of the API image took several minutes because TensorFlow is big.

Task 8: Integrating Flask and the API

What I did. I added `client.py`, which sends the image to the hosted API with `requests.post` (form field `image`, header `X-API-Key`, timeout 60 seconds) and returns the label, confidence, AI score and the model name from the JSON. In `app.py`, if the variable `API_URL` is set (in my `.env`: `API_URL=http://18.206.172.160`) the app sends images to the API and does not load any model on my laptop. If it is empty, the app uses the local model as in Task 4, so the Docker Compose stack of Task 6 still works. The Flask app itself still runs on localhost; only the prediction is remote.

Error handling. If the API returns 400 the user sees a message for that file. If the server is unreachable, times out or returns an error, the user sees "The prediction service is not reachable right now" with status 502. Tests with a fake API (`tests/test_client.py`) check both cases.

Multiple inputs. The upload form accepts several files at once (`multiple`). The view loops over the files, sends each one to the API, saves one row in the database for each one and shows one result card per image on the result page. A bad file shows its own message and does not stop the others. The 5 MB limit applies to the whole upload.

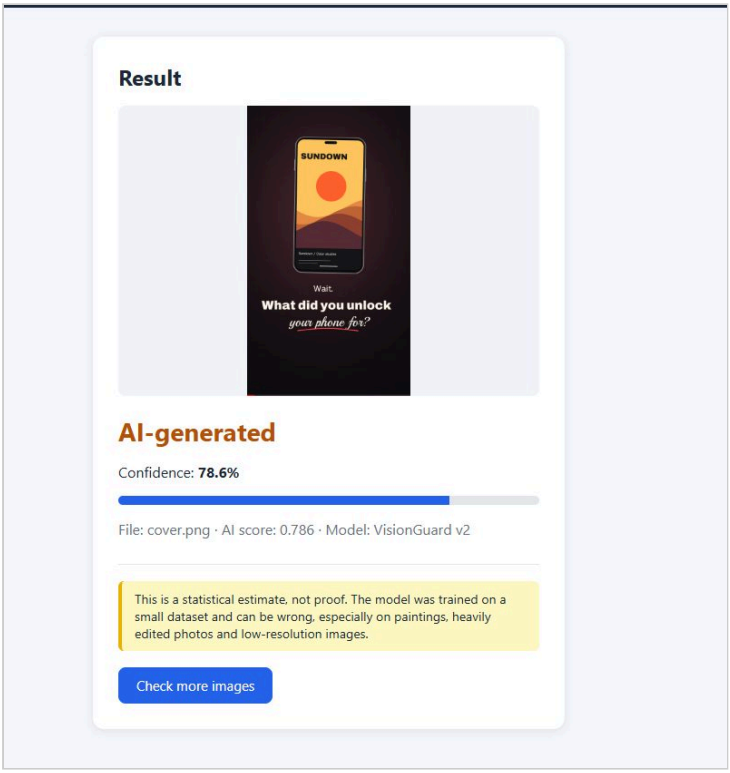


Figure 53. *A result returned by the hosted API and shown by the local Flask app (model VisionGuard v2).*

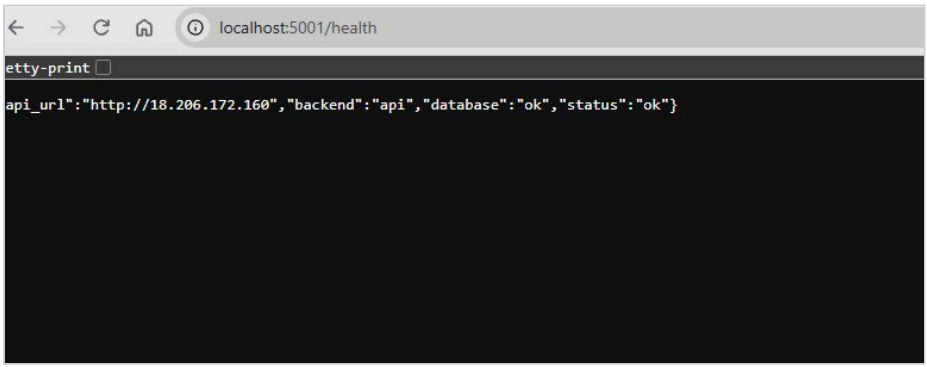
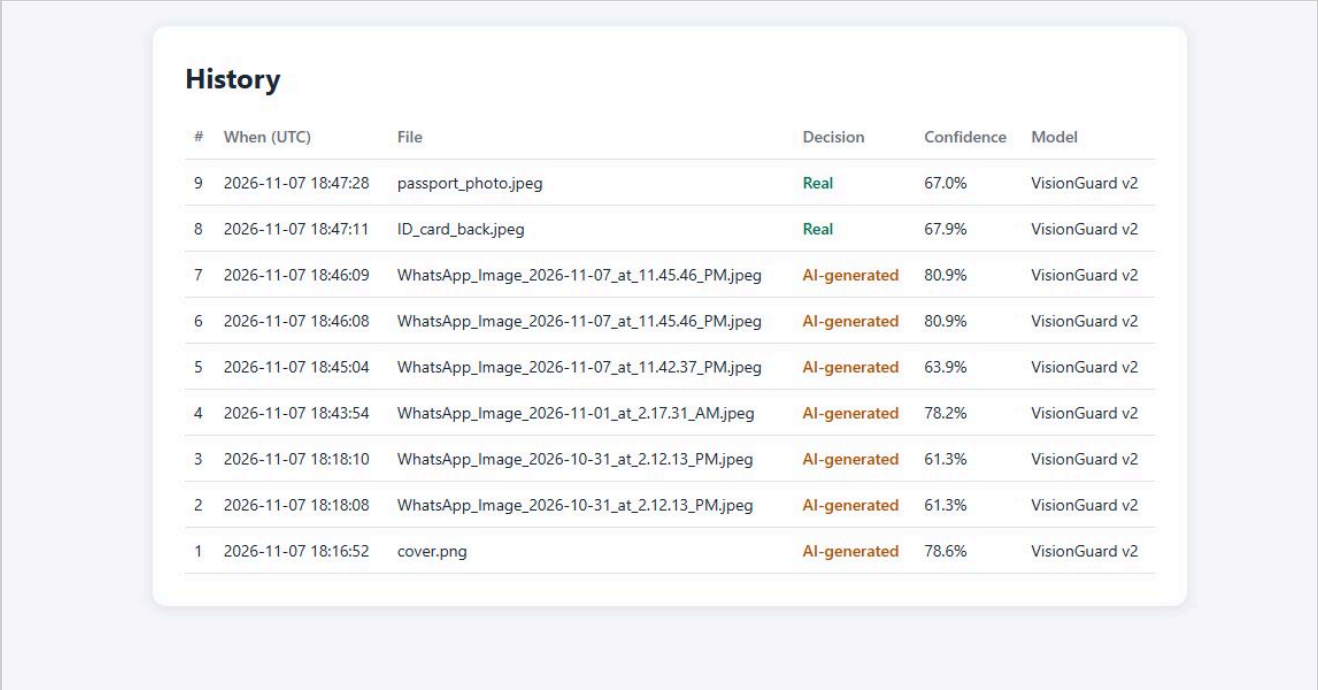


Figure 54. *Local Flask /health with backend "api" and the AWS address: proof that the app is using the hosted API.*

Database logs. Each call that returns a result adds a row, including the model name that the API reported. The history page below shows the rows (the model column says VisionGuard v2, the model that runs on AWS). Rows that are two seconds apart are images sent in quick succession.



History

#	When (UTC)	File	Decision	Confidence	Model
9	2026-11-07 18:47:28	passport_photo.jpeg	Real	67.0%	VisionGuard v2
8	2026-11-07 18:47:11	ID_card_back.jpeg	Real	67.9%	VisionGuard v2
7	2026-11-07 18:46:09	WhatsApp_Image_2026-11-07_at_11.45.46_PM.jpeg	AI-generated	80.9%	VisionGuard v2
6	2026-11-07 18:46:08	WhatsApp_Image_2026-11-07_at_11.45.46_PM.jpeg	AI-generated	80.9%	VisionGuard v2
5	2026-11-07 18:45:04	WhatsApp_Image_2026-11-07_at_11.42.37_PM.jpeg	AI-generated	63.9%	VisionGuard v2
4	2026-11-07 18:43:54	WhatsApp_Image_2026-11-01_at_2.17.31_AM.jpeg	AI-generated	78.2%	VisionGuard v2
3	2026-11-07 18:18:10	WhatsApp_Image_2026-10-31_at_2.12.13_PM.jpeg	AI-generated	61.3%	VisionGuard v2
2	2026-11-07 18:18:08	WhatsApp_Image_2026-10-31_at_2.12.13_PM.jpeg	AI-generated	61.3%	VisionGuard v2
1	2026-11-07 18:16:52	cover.png	AI-generated	78.6%	VisionGuard v2

Figure 55. *History page after sending several images through the hosted API.*

What went wrong. The first time I ran the app against the API every request ended with the 502 message. The `client.py` module read `API_URL` and `API_KEY` when it was imported, before the `.env` file was loaded, so it had no URL and no key. I made `client.py` load `.env` itself. Also my 4 tests started to fail because they picked up the real `API_URL` from my `.env`; the test setup now sets `API_URL` to empty. Port 5000 on my laptop was already used by another program, so I run the app on port 5001.

Conclusion and limits

- The system works end to end: data versioned with DVC, models tracked on DagsHub, the champion registered, a Flask app with a history database, a Docker Compose stack, CI, and a hosted API that the local app calls.
- The model is not strong: the best test ROC-AUC is 0.856 and about half of the real images are called AI at the threshold chosen for F1. The dataset is small (974 images) and the "real" class includes paintings and illustrations, which makes the problem hard and the labels noisy.
- With more time I would: use a higher threshold or show an "uncertain" band, collect more real photographs, add HTTPS to the API, use an Elastic IP or a domain name, try Grad-CAM to see what the model looks at, and run several seeds before comparing runs.

Files in the repository

File	Purpose
audit.py, eda.py, make_splits.py	Task 1: audit, figures, splits
data.py, preprocess.py, helpers.py	Input pipeline, shared preprocessing, callbacks and curves
train_cnn.py, train_transfer.py, evaluate.py, errors.py, tune.py	Task 2
train_tracked.py, register_model.py, download_champion.py, load_champion.py, toyrun/	Task 3
inference.py, app.py, db.py, client.py, templates/, static/	Tasks 4, 5 and 8
Dockerfile, docker-compose.yml, .dockerignore, .github/workflows/ci.yml, tests/	Task 6
api.py, Dockerfile.api, requirements-api.txt	Task 7 (hosted API)